Equivalence of Scott's Three Presentations of Domain Theory

Lars Warren Ericson[1]

[1]Catskills Research Company
[1]https://github.com/catskillsresearch/scott_models
[1]lars.ericson@catskillsresearch.com

August 19, 2026

ORCID: 0000-0001-8299-9361

Abstract

Dana Scott developed three presentations of the same class of domains: **continuous lattices** (1972), **neighbourhood systems** (PRG-19 / 1980–81), and **information systems** (1982). Each sibling formalization — `scott1972`, `scott1980`, `scott1982` — treats one presentation in isolation. This package (`ScottModels`) supplies the **bridge theorems**: order isomorphisms and constructivity audits showing that the three presentations determine the same domains (up to isomorphism), including products, separated sums, and function spaces at the information-system level, with a transport of 1972 Scott maps along the round presentation.

The Lean development is sorry-free. The **1980 ↔ 1982** bridges and the round continuous lattice corner target `#print axioms ⊆ {propext, Quot.sound}`. Classical choice appears where Scott's 1972 topology is unavoidable (algebraic ⇒ continuous; ScottMap conjugation) and in the trichotomy for separated sums.

1 Introduction

Scott notes in the 1982 ICALP paper that neighbourhood systems and information systems are equivalent "in a precise sense." The mathematical folklore is stronger still: all three presentations carve out the same class of domains, related by ideal completion and the Scott topology. Until the bridges are checked in a proof assistant, that claim lives in the gap between three separately formalized libraries.

This article closes that gap. We do **not** re-prove Scott's internal theorems; we import the finished sibling packages and build cross-presentation maps:

Presentation	Lean package	Characteristic object
Continuous lattices [**Sco72**]	`scott1972`	`IsContinuousLattice D`, way-below $\ll$, `ScottMap`
Neighbourhood systems [**Sco81**]	`scott1980`	`NeighborhoodSystem`, filters as domain elements
Information systems [**Sco82**]	`scott1982`	`InfoSys`, elements as consistent closed token sets

The main subtlety is that a naïve reading of "domains = filters of neighbourhoods" overstates the continuous-lattice corner: for a continuous lattice `D`, the filter `{⇑a | a ≤ x}` has retract `x`

but properly contains the principal round filter `{⇑a | a ≪ x}`. The correct identification is `D ≃o RoundFilter`, not raw `||`.

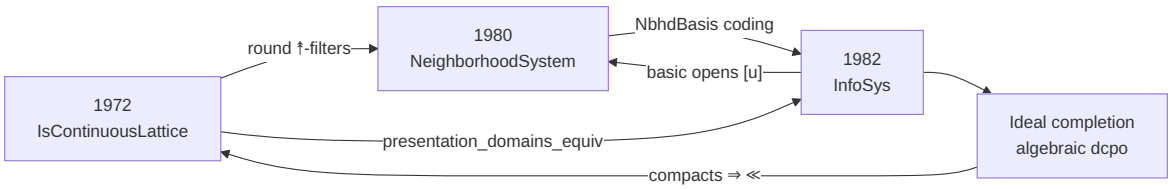

Figure 1: Introduction.

We prove **order isomorphisms of domains** (and of the named construction objects), not a 2-categorical equivalence of the full categories of continuous lattices / neighbourhood systems / information systems with all morphisms. Morphisms are linked where the sibling packages already supply them:

- approximable maps ↔ Scott-continuous maps on `|A|` (Factoid 4.6);
- Scott maps of continuous lattices ↔ their conjugates on the round presentation.

A full functorial equivalence (preserving products, exponentials, and inverse limits simultaneously across all three presentations) would require additional coherence theorems.

---

2 Catalog of bridge theorems

Theorem	Direction	Lean module
`continuousLattice_to_neighborhoodSystem`	1972 → 1980	`ContinuousLatticeToNeighborhood.lean`
`neighborhoodSystem_to_infoSys`	1980 → 1982	`NeighborhoodToInfoSys.lean`
`infoSys_to_neighborhoodSystem`	1982 → 1980	`InfoSysToNeighborhood.lean`
`infoSys_to_idealCompletion`	1982 → algebraic	`InfoSysToIdealCompletion.lean`
`idealCompletion_to_continuousLattice`	algebraic → 1972	`IdealCompletionToContinuousLattice.lea`
`presentation_domains_equiv`	three-way	`PresentationDomains.lean`
`infoSys_constructions_equiv`	constructions	`InfoSysConstructions.lean`, `ScottMapBridge.lean`

The sibling packages are **finished dependencies**, not work items of this paper: `scott1972`, `scott1980`, and `scott1982` (information systems through Factoid 8.4 / domain equations). They are vendored in `vendor/` at frozen SHAs (see `vendor/FROZEN.txt` / `lakefile.toml`); the remotes remain the per-paper homes. The Palomar statement of record is the presentation-bridge family in the catalog below (`Challenge.lean` / `comparator.json`), not a formalization of

the three papers. Figure~2 is left-to-right: sibling packages on the left, local `ScottModels/` modules on the right, with arrows for **direct** imports (transitive imports inside the siblings are omitted). `Equivalence` re-exports the bridges; only the `ScottMapBridge` → `Equivalence` edge is drawn to avoid clutter. Root `ScottModels.lean` also opens 1972 `ContinuousLattice.Specialization`, 1980 `Neighborhood.Basic`, and 1982 `InfoSys`.

---

3 Proof notes

3.1 Continuous lattices → neighbourhood systems

Claim. For a continuous lattice `D`, the sets `⇑a = {z | a ≪ z}` form a `NeighborhoodSystem` on token type `D`, and under `IsContinuousLattice` one has an order isomorphism `D ≃o RoundFilter`.

Construction. `wayBelowUp a` is upward closed; `⇑⊥ = univ`; `⇑a ∩ ⇑b = ⇑(a ⊔ b)` (interpolation / directedness of way-below). Principal filters `toFilter x = {⇑a | a ≪ x}` are round: membership of `⇑a` yields, by interpolation in a continuous lattice, some `b` with `a ≪ b ≪ x`, hence `⇑b ∈ toFilter x`.

Why raw `||` fails. The larger filter `{⇑a | a ≤ x}` still has `ofFilter = x` (supremum of codes), but is not equal to `toFilter x` whenever there are elements way-below strictly below `x`. Roundness (`⇑a ∈ f ⇒ ∃ b, a ≪ b ∧ ⇑b ∈ f`) cuts exactly to the image of `toFilter`.

Axioms. `domainOrderIso : D ≃o RoundFilter` audits to `{propext, Quot.sound}`.

3.2 Neighbourhood systems → information systems

Claim. A neighbourhood system equipped with a decidable exhaustive coding `NbhdBasis ι α` of its neighbourhood family induces an `InfoSys` on tokens `ι` with `||` `≃o` the InfoSys domain.

Construction. Tokens are neighbourhood indices. Consistency of a finite `u ⊆ ι` is membership of `⋂_{i∈u} nbhd i` in (empty intersection = master set `Δ`). Entailment `u ⊢ j` means the intersection is a neighbourhood contained in `nbhd j`. Filters of the neighbourhood system correspond to InfoSys elements via the coding.

Constructivity. `DecidableEq ι` is required so `InfoSys` can use the choice-free `Finset` prelude from `scott1982`. The proof of `ent_con` avoids classical `by_cases` / `em`. Axioms ⊆ `{propext, Quot.sound}`.

3.3 Information systems → neighbourhood systems

Claim. For an information system `A`, the basic opens `[u] = {x ∈ |A| | ↑u ⊆ x}` (`u ∈ Con`) form a `NeighborhoodSystem` on `|A|`, and filters recover elements: `|A|` `≃o` the filter domain.

Proof note. Scott's Factoid 4.6 supplies the basic-open vocabulary. The Lean proof initially pulled `Classical.choice` through `simp` on `basicOpen_empty` / finset unions; those simps were removed so the footprint stays `{propext, Quot.sound}`. Together with §3.2 this is the constructive **1980 ↔ 1982** equivalence under coding.

3.4 Information systems → ideal completion

Claim. `|A| ≃o Ideal (FiniteElement A)`, where finite elements are closures of consistent finsets (`Factoid 3.5`).

Construction. `toIdeal x` is the ideal of finite approximants of `x`; `ofIdeal` takes directed suprema of finite elements (1982 Factoids 4.4–4.5: `directedSup`, `eq_directedSup_finiteApproximants`, `compact_closure`). Axioms ⊆ `{propext, Quot.sound}`.

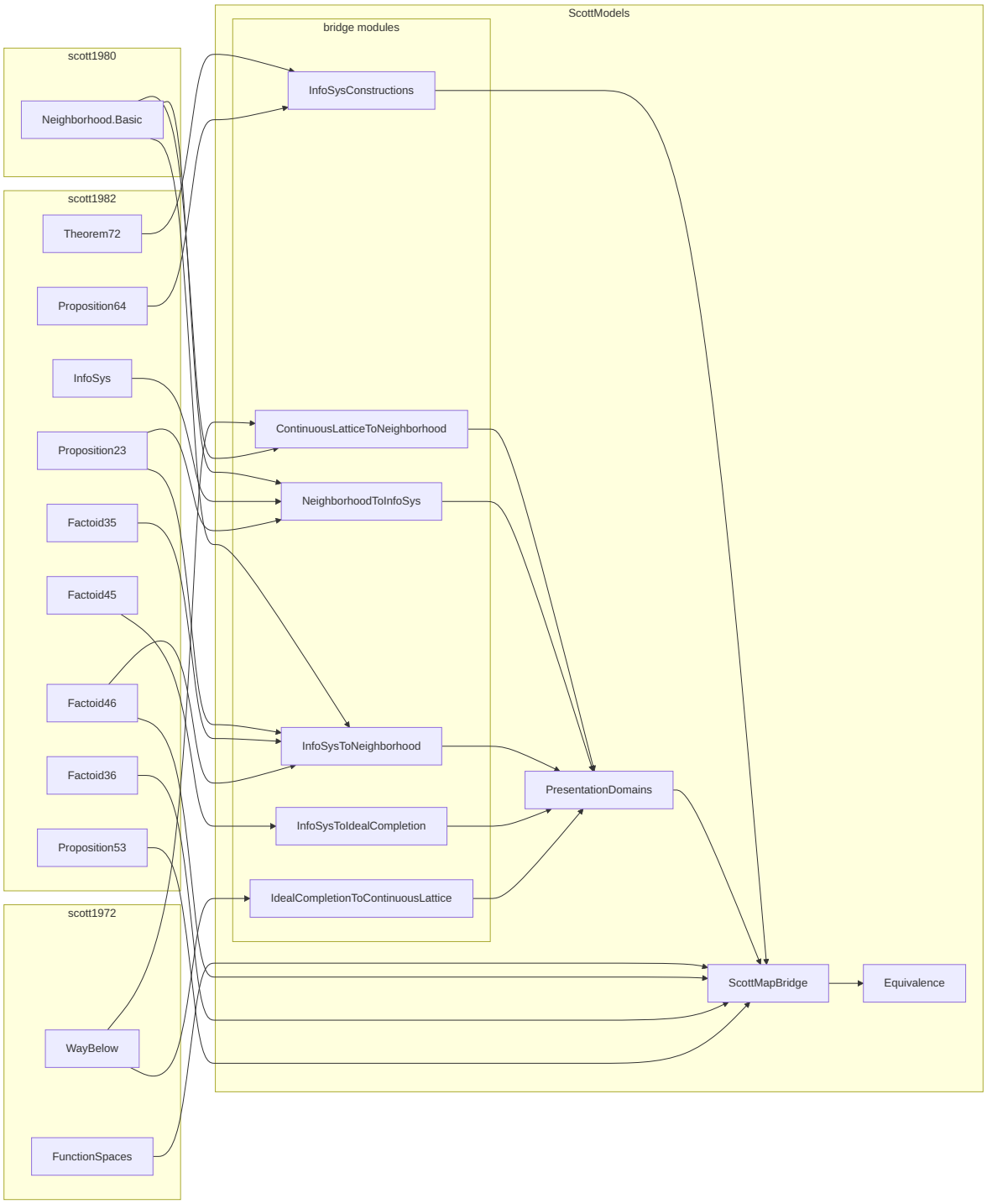

Figure 2: Catalog of bridge theorems.

3.5 Algebraic complete lattices → continuous lattices

Claim. If every element of a complete lattice is the directed supremum of compact elements below it (`IsAlgebraicLattice`), then `IsContinuousLattice` holds.

Proof note. Order-theoretic compactness implies `Set.Ici k` is Scott-open, hence `k` $\ll$ `y` whenever `k` $\leq$ `y`. Algebraicity then yields `y = ⊔{k compact | k ≤ y}` $\subseteq$ `⊔{x | x ≪ y}`. This is the **classical frontier**: Scott's $\ll$ is defined topologically in `scott1972`, so the footprint inherits that classicality even though the order argument is elementary.

3.6 Three-presentation equivalence (`presentation_domains_equiv`)

Claim. Under `IsContinuousLattice D` and `DecidableEq D`,

```
D ?o RoundFilter ?o RoundInfoSysElement
```

where `RoundInfoSysElement` is the subtype of elements of the InfoSys coded by `wayBelowNbhdBasis` (tokens = elements of `D`, neighbourhoods = ⇑`a`) that correspond to round filters. An extended form routes through the ideal-completion subtype.

Glue. `wayBelowNbhdBasis` packages the ⇑-system as an `NbhdBasis`; `roundFilter_infoSys_iso` transports roundness along `NbhdBasis.domainOrderIso`; `presentation_domains_equiv` is the composite with `continuousLattice_roundFilter_iso`.

What is not claimed. Raw `||` and the full InfoSys domain `|A|` remain properly larger than `D`. The equivalence is on the **round** corner that matches continuous lattice points.

Axioms. $\subseteq$ `{propext, Quot.sound}`.

3.7 Constructions (`infoSys_constructions_equiv`)

Products

`|A| × |B| ≃o |A × B|` via 1982 `pairElements` / `fstMap` / `sndMap` (Prop 6.2). Choice-free; axioms $\subseteq$ `{propext, Quot.sound}`.

Separated sums

`WithBot (|A| ⊕ |B|) ≃o |A + B|` via `inl` / `inr` classify-and-assemble (Prop 6.4). Trichotomy on token polarity uses classical case-split (`Classical.choice` in the footprint).

Function spaces

`ApproximableMap A B ≃o |A → B|` packages Theorem 7.2 (`approxMap_toElement` / `element_toApproxMap`) with pointwise `Le` as `PartialOrder`. Axioms $\subseteq$ `{propext, Quot.sound}`.

Factoid 4.6 bridge

`ApproximableMap A B ≃o ScottContinuous A B` via `toScottContinuous` / `ofScottContinuous`, using Prop 5.3(v) (`rel_iff_closure_le`) and `closure_le_element` for the round-trip on relations. Constructive: `{propext, Quot.sound}`.

ScottMap conjugation

A 1972 `ScottMap D E` conjugates along any pair of order isomorphisms `ιD : D ≃o D'`, `ιE : E ≃o E'` to a pointwise-ordered map `D' → E'`. Specializing to `continuousLattice_roundFilter_iso` / `presentation_domains_equiv` yields `scottMap_roundFilter_iso` and `scottMap_roundInfoSys_iso`. The conjugation itself is order-theoretic; the footprint includes `Classical.choice` because `ScottMap` is defined via Scott continuity in `scott1972`.

Out of scope (documented). Identifying `ApproximableMap` on the coded ⇑-InfoSys with `ScottMap D E` (needs roundness preservation of approximable maps), and relating `wayBelow(D × E)` to the InfoSys product of factors (cylinder basis).

---

4 Constructivity summary

Bridge	Footprint	Notes
Nbhd ↔ InfoSys (both directions)	`{propext, Quot.sound}`	Decidable coding; avoid classical `simp` traps
CL ↔ RoundFilter	`{propext, Quot.sound}`	Roundness is order-theoretic
InfoSys ↔ Ideal	`{propext, Quot.sound}`	Factoids 4.4–4.5
Algebraic ⇒ CL	classical	1972 topological $\ll$
Product / function space (1982)	`{propext, Quot.sound}`	
Separated sum (1982)	+ `Classical.choice`	Token polarity trichotomy
Factoid 4.6 ApproxMap ↔ ScottContinuous	`{propext, Quot.sound}`	
ScottMap conjugation	+ `Classical.choice`	Via 1972 `ScottMap`

Target discipline (from the 1982 package): prefer constructive proofs wherever Scott's 1982 text emphasizes constructivity; call out classical frontiers explicitly rather than hiding choice in automation.

---

5 Worked example — S-expressions / trees

Lean packaging: `WorkedExampleSExpr.lean`.

5.1 Overview

The bridges of §§2–3 are abstract. This section fixes **one** concrete domain — Scott's S-expression / tree equation `T ≅ A + (T × T)` (**[Sco82]**, Factoid 8.1) over the $\mathbb{N}$ lower-bound atom system (Factoid 2.4) — and asks what that domain looks like in each paradigm:

Paradigm	What an element of `T` is
1982	a consistent, deductively closed set of tree tokens
1980	a filter of basic opens `[u]` on that token domain
1972	a point of an algebraic continuous lattice (ideal of finite elements)

We build the domain natively as an information system (§5.2), then **read the same carrier** as a neighbourhood system (§5.3) and as a continuous lattice (§5.4). Section 5.5 exhibits the order isomorphisms for this instance — so that, on S-expressions,

`T_1982` $\simeq_o$ `T_1980` $\simeq_o$ `Ideal(K(T))`and`Ideal(K(T))` is a continuous lattice,

hence transitively `T_1982` $\simeq_o$ `T_1980` and (via the algebraic ⇒ continuous frontier) the 1972 presentation of the same domain. Order isomorphism is an equivalence relation, so the three presentations determine one domain up to $\simeq_o$. Section 5.6 records two extras that live on top of the carriers: the domain equation at the level of domains, and identity as a morphism in both the approximable-map and Scott-continuous languages.

5.2 Information system (1982)

Atoms are propositions `n` $\leq$ `x` on $\mathbb{N}$ (`lowerBoundSystem`). The tree system `SexSys := treeSystem lowerBoundSystem` has inductive tokens `TreeToken` $\mathbb{N}$ (`bot` / `atom` / `pairL` / `pairR`). Consistency and entailment are Scott's sum×product clauses; Factoid 8.1 records that this is literally the information system of the right-hand side `A + (T × T)`:

- `SexRhs = sumSystem A (productSystem T T)` (`sexRhs_eq_sum_product`);
- token unfolding `treeUnfold` sends `atom n` to the left summand (`sexUnfold_atom`).

Finite elements include singleton atom closures `sexAtom n =` for `u = {atom n}`. Elements of `|T|` are the consistent closed sets of tokens — the 1982 presentation of the S-expression domain.

5.3 Neighbourhood system (1980)

From `|T|` one obtains a neighbourhood system whose basic opens are `[u] = {x ∈ |T| | ↑u ⊆ x}`. In this paradigm an element of the domain is a **filter** of such opens (not a raw set of tokens). Nothing new is invented: it is the image of `SexSys` under `InfoSysToNeighborhood`. The concrete iso of §5.5 below is exactly "read each token-element as its filter of basic opens."

5.4 Continuous lattice (1972)

Finite elements `K(T)` are the closures of consistent finsets. In the algebraic style that feeds Scott's 1972 theory, a domain element is an **ideal** of `K(T)` — the ideal completion of the compact basis. That poset of ideals is an algebraic complete lattice, hence a continuous lattice (`IsAlgebraicLattice` ⇒ `IsContinuousLattice`, §3.5). This is the 1972 presentation of the same S-expression domain: points ordered by inclusion of ideals, with way-below recovered from the compact basis.

We do not claim a choice-free `D` $\simeq$`o RoundFilter` story built from trees alone; the round ⇑-filter identification of §3.1 applies once one is already in the continuous-lattice setting.

5.5 Equivalence for this instance

Specializing the bridges of §3 to `SexSys` yields order isomorphisms of carriers:

- `sexNeighborhoodIso : |T|` $\simeq$`o` filters of `[u]` (`InfoSysToNeighborhood.domainOrderIso`) — **1982 $\simeq$o 1980**;
- `sexIdealIso : |T|` $\simeq$`o Ideal (FiniteElement T)` (`InfoSysToIdealCompletion.domainOrderIso`) — **1982 $\simeq$o algebraic / 1972 carrier**;
- `sexNeighborhoodIdealIso` (`neighborhood_ideal_iso`) — the constructive triangle closing **1980 $\simeq$o Ideal(K(T))** without going back through tokens.

Composing any two edges recovers the third: isomorphism is transitive, so on this example the three presentations are pairwise equivalent. The only classical step on the 1972 corner is the algebraic ⇒ continuous implication already flagged in §3.5 / §4; the neighbourhood ↔ information ↔ ideal triangle audits to `{propext, Quot.sound}`.

5.6 Domain equation and morphisms

Beyond identifying the carriers, the same example exercises constructions and maps.

Domain equation. This article's product and separated-sum isos lift Factoid 8.1 from tokens to domains:

`sexDomainEquationIso : WithBot (|A| ⊕ (|T| × |T|)) ≃o |A + (T × T)|`

(classical footprint only through the sum trichotomy). So the S-expression equation holds not only as information systems but as ordered domains.

Morphisms. The identity approximable map `idMap T` (Prop 5.4) transports along Factoid 4.6 to a Scott-continuous endomap `sexIdScottContinuous` with `toFun = id` (`sexId_toElement`) — the same endomap, named once in each morphism language.

Axioms. Object-level neighbourhood / ideal isos and Factoid 4.6 for `idMap` audit to `{propext, Quot.sound}` up to the usual classical sum iso in `sexDomainEquationIso`.

---

6 Build

```
lake exe cache get
lake build ScottModels
```

Pinned: Lean / mathlib v4.30.0; sibling packages via in-tree `vendor/` path deps (frozen SHAs in `vendor/FROZEN.txt`) (`scott1972`, `scott1980`, `scott1982`). Palomar compared family: `Challenge.lean`, `Solution.lean`, `comparator.json`; paper of record for that packaging is `view.pdf`.

Acknowledgments (Dana Scott, AI tool cards, artifact URL) are injected before References when building `arxiv.tex` via `scripts/ai_model_cards.py` — they are not kept in this file.

Rebuild this document (PDF with complete Lean source appendix, one subsection per module):

```
bash scripts/build_arxiv_pdf.sh            # expand Lean  ->  tex  ->  arxiv.pdf
bash scripts/build_arxiv_pdf.sh --pdf-only # PDF only when arxiv.tex already current
```

---

7 Acknowledgments

- **Dana Scott** — *Continuous Lattices* [**Sco72**], *Lectures on a Mathematical Theory of Computation* (PRG-19) [**Sco81**], and *Domains for Denotational Semantics* [**Sco82**], the three presentations this development relates.

7.1 AI-assisted development

The human author retains sole responsibility for the mathematical content, the choice of formalization route, and every formal claim in this work. Following standard publisher practice (e.g., COPE guidance on authorship and AI tools [**COPE24**]), **no large language model is listed as a co-author** — authorship implies an accountability that automated systems cannot bear.

Because this development may borrow Lean and proof patterns from the sibling formalizations `scott1972` [**SR72**], `scott1980` [**ER80**], and `scott1982` [**SR82**], **every model in the registry is treated as used** for acknowledgement purposes. We gratefully acknowledge assistance from the following tools (auto-generated from `scripts/ai_model_cards.py` when building `arxiv.tex`):

- **Cursor [Cur26]** — agent-assisted editing in the Cursor IDE: bridge theorems relating Scott's 1972 / 1980 / 1982 presentations in Lean 4 / mathlib, `lake build` repair, drafting this narrative (`arxiv.md`), and tracking the formalized inventory. Generated Lean was provisional until it compiled under the pinned toolchain. Also used in the sibling formalizations whose portable work may be copied into this repo.
- **Cursor Composer 2.5 Fast [Cmp25]** — routine multi-step work: module scaffolding, dependency-ordered wiring of `ScottModels/`, documentation and Mermaid blueprints, and medium proof obligations where the strategy was already fixed (`composer-2.5`). Also used in sibling repos whose portable Lean may be adapted here.
- **Cursor Grok 4.5 [Grk45]** — primary agent model for the scott_models sessions: bridge theorems (`presentation_domains_equiv`, round filters, constructions), inventory design in `arxiv.md`, constructivity audits, and adaptation of portable patterns from prior Scott formalizations. Jointly trained by SpaceXAI and Cursor; used here via the Cursor agent environment.
- **Anthropic Claude Sonnet 5 (medium reasoning) [Son26]** — day-to-day formalization and proof-engineering in Cursor at the medium reasoning tier in sibling Scott formalizations (and available for this repo): inventory and narrative maintenance, module wiring, and medium-complexity Lean obligations where the proof strategy was already fixed. Listed because portable prior work may be copied into scott_models.
- **Anthropic Claude Opus 4.8 (high reasoning) [Ant26]** — selective use for the heaviest proof work in sibling formalizations (e.g. continuous lattices and PRG-19). Every emitted proof term was checked by the Lean kernel. Listed because portable prior work may be copied into scott_models.
- **Google Gemini 3.5 Flash [Gem25]** — exploratory passes in sibling formalizations (typographic conventions, scope decisions). Listed because portable prior work may be copied into scott_models.

All definitions, constructivity audits, and final prose were reviewed by the human author, who takes full responsibility for them.

7.2 Artifact availability and reproducibility

The development is at github.com/catskillsresearch/scott_models. GitHub Actions CI checks out the three sibling packages as path dependencies and runs `lake build`; the library builds successfully on `main` (green status).

Locally:

```
lake exe cache get
lake build ScottModels
```

Sibling packages `scott1972`, `scott1980`, `scott1982` are Lake path dependencies (`lakefile.toml`). Session state lives in `HANDOFF.md`; `arxiv.md` is the durable inventory and proof narrative. Axiom audits: `#print axioms` on the blueprint-facing names (`presentation_domains_equiv`, `infoSys_product_domain_equiv`, `approximableMap_scottContinuous_equiv`, `scottMap_roundInfoSys_iso`, ...).

Build the arXiv PDF / submission zip (Lean Code appendix = GitHub hyperlinks):

```
bash scripts/build_arxiv_tex.sh      # arxiv.md  ->  arxiv.tex + figures/
bash scripts/build_arxiv_pdf.sh      # compile PDF + package dist/arxiv_submit.zip
# or: bash scripts/package_arxiv_submit.sh
```

List of Figures

8 References

- [**Sco72**] Dana Scott. *Continuous Lattices.* In F. W. Lawvere (ed.), *Toposes, Algebraic Geometry and Logic*, LNM 274, Springer, 1972.
- [**Sco81**] Dana Scott. *Lectures on a Mathematical Theory of Computation.* Technical Monograph PRG-19, Oxford University Computing Laboratory, 1981 (neighbourhood systems).
- [**Sco82**] Dana Scott. *Domains for Denotational Semantics.* ICALP 1982, LNCS 140, Springer, 1982.
- [**AJ94**] S. Abramsky and A. Jung. *Domain Theory.* In *Handbook of Logic in Computer Science*, Vol. 3, Oxford University Press, 1994.
- [**GHKLMS03**] G. Gierz et al. *Continuous Lattices and Domains.* Cambridge University Press, 2003.
- [**SR72**] Companion Lean formalization: `scott1972`.
- [**ER80**] Companion Lean formalization: `scott1980`.
- [**SR82**] Companion Lean formalization: `scott1982`.
- [**COPE24**] Committee on Publication Ethics (COPE). *Authorship and AI tools: COPE position statement.* 2024. `https://publicationethics.org/guidance/cope-position/authorship-and-ai-tools`
- [**Cur26**] Anysphere, Inc. *Cursor: AI-native code editor and agent environment.* `https://cursor.com` (accessed 2026).
- [**Cmp25**] Anysphere, Inc. *Composer 2.5.* Model announcement and documentation, `https://cursor.com/blog/composer-2-5`; model card as integrated in Cursor, `https://cursor.com/docs/models` (accessed 2026).
- [**Grk45**] SpaceXAI and Anysphere, Inc. *Grok 4.5.* Model documentation, `https://docs.x.ai/developers/models/grok-4.5`; Cursor announcement, `https://cursor.com/blog/grok-4-5` (accessed 2026).
- [**Son26**] Anthropic. *Claude Sonnet 5* (medium reasoning variant). System card, `https://www.anthropic.com/claude-sonnet-5-system-card`; model documentation as integrated in Cursor, `https://cursor.com/docs/models` (accessed 2026).
- [**Ant26**] Anthropic. *Claude Opus 4.8* (high thinking/reasoning variant). System card and announcement, `https://www.anthropic.com/news/claude-opus-4-8`; model documentation as integrated in Cursor, `https://cursor.com/docs/models/claude-opus-4-8` (accessed 2026).
- [**Gem25**] Google DeepMind. *Gemini 3.5 Flash.* Technical documentation and model cards. `https://ai.google.dev/gemini-api/docs/models` (accessed 2026).

A Complete Lean source

Role	File	Lines
Root import graph	`ScottModels.lean`	20
Bridge	`ScottModels/NeighborhoodToInfoSys.lean`	267
Bridge	`ScottModels/InfoSysToNeighborhood.lean`	242
Bridge	`ScottModels/ContinuousLatticeToNeighborhood.lean`	227
Bridge	`ScottModels/InfoSysToIdealCompletion.lean`	183
Bridge	`ScottModels/IdealCompletionToContinuousLattice.lean`	78
Bridge / packaging	`ScottModels/PresentationDomains.lean`	168
Bridge / packaging	`ScottModels/InfoSysConstructions.lean`	469
Bridge / packaging	`ScottModels/ScottMapBridge.lean`	190
Bridge / packaging	`ScottModels/WorkedExampleSExpr.lean`	139
Bridge / packaging	`ScottModels/Equivalence.lean`	41

Total: 11 files, 2024 lines of Lean.

Files appear in `ScottModels.lean` import order. Each block is a verbatim copy of the repository file at generation time.

A.1 ScottModels.lean

20 lines.

Lean 4 source

```
/-
Copyright (c) 2026  Lars Warren Ericson.  All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github:  https://github.com/catskillsresearch/scott_models
-/

import Scott1972.ContinuousLattice.Specialization
import Scott1980.Neighborhood.Basic
import Scott1982.InfoSys
import ScottModels.NeighborhoodToInfoSys
import ScottModels.InfoSysToNeighborhood
import ScottModels.ContinuousLatticeToNeighborhood
import ScottModels.InfoSysToIdealCompletion
import ScottModels.IdealCompletionToContinuousLattice
import ScottModels.PresentationDomains
import ScottModels.InfoSysConstructions
import ScottModels.ScottMapBridge
import ScottModels.WorkedExampleSExpr
import ScottModels.Equivalence
```

A.2 ScottModels/NeighborhoodToInfoSys.lean

267 lines.

Lean 4 source

```
/-
Copyright (c) 2026  Lars Warren Ericson.  All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
```

```
Github:  https://github.com/catskillsresearch/scott_models
-/

import Mathlib.Data.Finset.Fold
import Mathlib.Order.Hom.Basic
import Scott1980.Neighborhood.Basic
import Scott1982.InfoSys
import Scott1982.Proposition23

/-!
# Neighbourhood systems  ->  information systems (1980  ->  1982)

Scott (1982) notes that neighbourhood systems and information systems are equivalent
in a precise sense. This module realises one direction: given a neighbourhood system
presented with a **decidable index of its neighbourhoods** (a constructive coding of
`?`), build an information system on those indices whose domain is order-isomorphic
to the original filter domain.

Tokens of the information system are indices of neighbourhoods; consistency is
membership of the finite intersection in `?` (Scott?s empty intersection = `Delta`);
entailment is ?the intersection is a neighbourhood and is contained in the target.?
-/

namespace ScottModels

open Scott1980.Neighborhood
open Scott1982

/-- A neighbourhood system with a decidable exhaustive index of its neighbourhoods.
`DecidableEq ?` is required so the induced information system can use choice-free
`Finset` operations (`InfoSys` tokens). -/
structure NbhdBasis (? a : Type*) [DecidableEq ?] where
  system : NeighborhoodSystem a
  nbhd : ?  ->  Set a
  nbhd_mem : forall  i, system.mem (nbhd i)
  exhaustive : forall  {X : Set a}, system.mem X  ->  exists  i, nbhd i = X
  botIdx : ?
  botIdx_eq : nbhd botIdx = system.master

namespace NbhdBasis

variable {? a : Type*} [DecidableEq ?] (B : NbhdBasis ? a)

/-- Finite intersection of coded neighbourhoods, with Scott?s convention `?{} = Delta`. -/
def interOf (u : Finset ?) : Set a :=
  u.fold (*  I  *) B.system.master B.nbhd

@[simp] theorem interOf_empty : B.interOf ({} : Finset ?) = B.system.master := rfl

theorem interOf_insert {a : ?} {s : Finset ?} (ha : a  notin  s) :
    B.interOf (insert a s) = B.nbhd a  I  B.interOf s :=
  Finset.fold_insert (op := (*  I  *)) ha

theorem nbhd_subset_master (i : ?) : B.nbhd i  subseteq  B.system.master :=
  B.system.sub_master (B.nbhd_mem i)

theorem interOf_singleton (a : ?) : B.interOf ({a} : Finset ?) = B.nbhd a := by
  simp [interOf, Finset.fold_singleton, Set.inter_eq_left.mpr (B.nbhd_subset_master a)]

/-- Membership in the folded intersection. -/
theorem mem_interOf {u : Finset ?} {x : a} :
    x  in  B.interOf u  <->  x  in  B.system.master  /\  forall  i  in  u, x  in  B.nbhd i :=
    by
```

```
  induction u using Finset.induction with
  | empty =>
    simp [interOf_empty]
  | insert a s ha ih =>
    rw [B.interOf_insert ha, Set.mem_inter_iff, ih]
    constructor
    * rintro <ha', hm, hs>
      refine <hm, fun i hi => ?_>
      rcases Finset.mem_insert.mp hi with rfl | hi
      * exact ha'
      * exact hs i hi
    * rintro <hm, hall>
      exact <hall a (Finset.mem_insert_self a s), hm,
        fun i hi => hall i (Finset.mem_insert_of_mem hi)>

theorem interOf_subset_nbhd {u : Finset ?} {i : ?} (hi : i  in  u) :
    B.interOf u  subseteq  B.nbhd i := fun _ hx => (B.mem_interOf.mp hx).2 i hi

theorem interOf_subset_master (u : Finset ?) : B.interOf u  subseteq  B.system.master :=
  fun _ hx => (B.mem_interOf.mp hx).1

/-- Larger index sets give smaller intersections. -/
theorem interOf_anti {u v : Finset ?} (h : u  subseteq  v) : B.interOf v  subseteq  B.interOf u
     := by
  intro x hx
  exact B.mem_interOf.mpr <(B.mem_interOf.mp hx).1, fun i hi => (B.mem_interOf.mp hx).2 i (h hi
    )>

/-- If a neighbourhood sits below `interOf u`, then `interOf u  in  ?`. -/
theorem interOf_mem_of_lower_bound {u : Finset ?} {Z : Set a}
    (hZ : B.system.mem Z) (hsub : Z  subseteq  B.interOf u) : B.system.mem (B.interOf u) := by
  induction u using Finset.induction with
  | empty =>
    simpa [interOf_empty] using B.system.master_mem
  | insert a s ha ih =>
    rw [B.interOf_insert ha] at hsub  |-
    have hs : B.system.mem (B.interOf s) := ih (hsub.trans Set.inter_subset_right)
    exact B.system.inter_mem (B.nbhd_mem a) hs hZ hsub

/-- A filter containing every `nbhd i` for `i  in  u` contains `interOf u`. -/
theorem filter_mem_interOf (x : B.system.Element) {u : Finset ?}
    (hu : forall  i  in  u, x.mem (B.nbhd i)) : x.mem (B.interOf u) := by
  induction u using Finset.induction with
  | empty =>
    simpa [interOf_empty] using x.master_mem
  | insert a s ha ih =>
    rw [B.interOf_insert ha]
    exact x.inter_mem (hu a (Finset.mem_insert_self a s))
      (ih fun i hi => hu i (Finset.mem_insert_of_mem hi))

/-- **`neighborhoodSystem_to_infoSys`.** -/
def toInfoSys : InfoSys ? where
  bot := B.botIdx
  Con := {u | B.system.mem (B.interOf u)}
  Ent := fun u a => B.system.mem (B.interOf u)  /\  B.interOf u  subseteq  B.nbhd a
  con_subset := by
    intro u v hu hv
    exact B.interOf_mem_of_lower_bound hu (B.interOf_anti hv)
  con_sing := by
    intro a
    simpa [Set.mem_setOf_eq, B.interOf_singleton] using B.nbhd_mem a
  ent_con := by
    intro u a <hu, hsub>
```

```
    have h_1 : B.interOf (insert a u)  subseteq  B.interOf u :=
      B.interOf_anti (Finset.subset_insert a u)
    have h_2 : B.interOf u  subseteq  B.interOf (insert a u) := by
      intro x hx
      refine B.mem_interOf.mpr <(B.mem_interOf.mp hx).1, fun i hi => ?_>
      rcases Finset.mem_insert.mp hi with rfl | hi
      * exact hsub hx
      * exact (B.mem_interOf.mp hx).2 i hi
    have heq : B.interOf (insert a u) = B.interOf u := Set.Subset.antisymm h_1 h_2
    simpa [Set.mem_setOf_eq, heq] using hu
  ent_bot := by
    intro u hu
    refine <hu, ?_>
    intro x hx
    rw [B.botIdx_eq]
    exact B.interOf_subset_master u hx
  ent_refl := by
    intro u a hu ha
    exact <hu, B.interOf_subset_nbhd ha>
  ent_trans := by
    intro u v c hv _hu hall <_, hsub>
    refine <hv, ?_>
    intro x hx
    have hxu : x  in  B.interOf u :=
      B.mem_interOf.mpr <B.interOf_subset_master v hx, fun i hi => (hall i hi).2 hx>
    exact hsub hxu

/-! ## Domain isomorphism -/

/-- Filter  ->  InfoSys element. -/
def toElement (x : B.system.Element) : B.toInfoSys.Element where
  carrier := {i | x.mem (B.nbhd i)}
  consistent := fun Y hY => x.sub (B.filter_mem_interOf x hY)
  closed := by
    intro Y a hY <_, hsub>
    exact x.up_mem (B.filter_mem_interOf x hY) (B.nbhd_mem a) hsub

/-- InfoSys element  ->  filter (upward closure of its coded neighbourhoods). -/
def ofElement (e : B.toInfoSys.Element) : B.system.Element where
  mem X := B.system.mem X  /\  exists  i  in  e.carrier, B.nbhd i  subseteq  X
  sub h := h.1
  master_mem := by
    refine <B.system.master_mem, B.botIdx, ?_, by rw [B.botIdx_eq]>
    have hEnt : B.toInfoSys.Ent {} B.botIdx := B.toInfoSys.ent_bot (InfoSys.con_empty _)
    exact e.closed {} B.botIdx (fun _ h => False.elim (Finset.notMem_empty _ h)) hEnt
  inter_mem := by
    intro X Y <hX, i, hi, hix> <hY, j, hj, hjy>
    have hCon : ({i, j} : Finset ?)  in  B.toInfoSys.Con :=
      e.consistent {i, j} (by
        intro x hx
        rcases Finset.mem_insert.mp hx with rfl | hx
        * exact hi
        * have : x = j := Finset.mem_singleton.mp hx
          exact this  |>  hj)
    have hInterMem : B.system.mem (B.interOf ({i, j} : Finset ?)) := hCon
    have hInterSub : B.interOf ({i, j} : Finset ?)  subseteq  X  I  Y := by
      intro x hx
      refine <hix (B.interOf_subset_nbhd (by simp) hx),
        hjy (B.interOf_subset_nbhd (by simp) hx)>
    refine <B.system.inter_mem hX hY hInterMem hInterSub, ?_>
    obtain <k, hk> := B.exhaustive hInterMem
    refine <k, ?_, ?_>
    * have hEnt : B.toInfoSys.Ent {i, j} k := <hInterMem, fun x hx => hk  |>  hx>
```

```
      exact e.closed {i, j} k (by
        intro x hx
        rcases Finset.mem_insert.mp hx with rfl | hx
        * exact hi
        * exact Finset.mem_singleton.mp hx  |>  hj) hEnt
    * intro x hx
      exact hInterSub (hk  |>  hx)
  up_mem := by
    intro X Y <hX, i, hi, hix> hY hXY
    exact <hY, i, hi, hix.trans hXY>

theorem toElement_carrier (x : B.system.Element) :
    (B.toElement x).carrier = {i | x.mem (B.nbhd i)} := rfl

theorem ofElement_mem (e : B.toInfoSys.Element) (X : Set a) :
    (B.ofElement e).mem X  <->  B.system.mem X  /\  exists  i  in  e.carrier, B.nbhd i
    subseteq  X :=
  Iff.rfl

theorem toElement_ofElement (e : B.toInfoSys.Element) :
    B.toElement (B.ofElement e) = e := by
  have hc : (B.toElement (B.ofElement e)).carrier = e.carrier := by
    ext i
    constructor
    * intro hi
      rcases (B.ofElement_mem e (B.nbhd i)).mp hi with <_, j, hj, hsub>
      have hEnt : B.toInfoSys.Ent {j} i := by
        refine <by simpa [B.interOf_singleton] using B.nbhd_mem j, ?_>
        simpa [B.interOf_singleton] using hsub
      exact e.closed {j} i (by
        intro x hx
        have : x = j := Finset.mem_singleton.mp hx
        exact this  |>  hj) hEnt
    * intro hi
      exact <B.nbhd_mem i, i, hi, subset_rfl>
  rcases e with <c, cons, clo>
  rcases hte : B.toElement (B.ofElement <c, cons, clo>) with <c', cons', clo'>
  have hc' : c' = c := by
    simpa [hte, toElement_carrier] using hc
  subst hc'
  rfl

theorem ofElement_toElement (x : B.system.Element) :
    B.ofElement (B.toElement x) = x := by
  refine NeighborhoodSystem.Element.ext (V := B.system) fun X => ?_
  constructor
  * intro <hX, i, hi, hsub>
    exact x.up_mem hi hX hsub
  * intro hX
    obtain <i, rfl> := B.exhaustive (x.sub hX)
    exact <x.sub hX, i, hX, subset_rfl>

/-- Order isomorphism between the neighbourhood-system domain and the induced
information-system domain. -/
def domainOrderIso : B.system.Element ?o B.toInfoSys.Element where
  toFun := B.toElement
  invFun := B.ofElement
  left_inv := B.ofElement_toElement
  right_inv := B.toElement_ofElement
  map_rel_iff' := by
    intro x y
    constructor
    * intro h X hX
```

```
      obtain <i, rfl> := B.exhaustive (x.sub hX)
      have hi : i ∈ (B.toElement x).carrier := hX
      have hi' : i ∈ (B.toElement y).carrier := h hi
      exact y.up_mem hi' (x.sub hX) subset_rfl
    · intro h i hi
      exact h (B.nbhd i) hi

end NbhdBasis

/-- Blueprint name: neighbourhood system (with decidable basis) → information system. -/
abbrev neighborhoodSystem_to_infoSys {? a : Type*} [DecidableEq ?] (B : NbhdBasis ? a) :
    InfoSys ? :=
  B.toInfoSys

end ScottModels
```

A.3 ScottModels/InfoSysToNeighborhood.lean

242 lines.

Lean 4 source

```
/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott_models
-/

import Mathlib.Order.Hom.Basic
import Scott1980.Neighborhood.Basic
import Scott1982.Factoid35
import Scott1982.Factoid46
import Scott1982.Proposition23

/-!
# Information systems → neighbourhood systems (1982 → 1980)

Scott (1982, §4): for `u ∈ Con`, the basic open
`⟦u⟧ = { x ∈ |A| | u ⊆ x }` yields a neighbourhood system on the domain `|A|`
whose filters recover the original elements. This is the converse direction to
`NeighborhoodToInfoSys`.
-/

namespace ScottModels

open Scott1980.Neighborhood
open Scott1982
open Scott1982.Constructive
open Scott1982.InfoSys

namespace InfoSysToNeighborhood

variable {a : Type*} [DecidableEq a] (A : InfoSys a)

theorem basicOpen_empty : A.basicOpen (∅ : Finset a) = (Set.univ : Set A.Element) := by
  ext x
  constructor
  · intro; trivial
  · intro _ a ha
    exact False.elim (Finset.notMem_empty a (Finset.mem_coe.1 ha))
```

```
theorem mem_basicOpen_singleton {a : a} {x : A.Element} :
    x  in  A.basicOpen ({a} : Finset a)  <->  a  in  x.carrier := by
  constructor
  * intro h
    exact h (Finset.mem_coe.2 (Finset.mem_singleton_self a))
  * intro ha b hb
    have hb' : b = a := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
    exact hb'  |>  ha

theorem funion_singleton_eq_insert (a : a) (s : Finset a) :
    ({a} : Finset a)  U ' s = insert a s := by
  ext x
  constructor
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact Finset.mem_insert.mpr (Or.inl (Finset.mem_singleton.mp h))
    * exact Finset.mem_insert_of_mem h
  * intro hx
    rcases Finset.mem_insert.mp hx with h | h
    * exact mem_funion.mpr (Or.inl (h  |>  Finset.mem_singleton_self a))
    * exact mem_funion.mpr (Or.inr h)

theorem basicOpen_singleton_inter (a : a) (s : Finset a) :
    A.basicOpen ({a} : Finset a)  I  A.basicOpen s = A.basicOpen (insert a s) := by
  rw [basicOpen_inter A, funion_singleton_eq_insert]

theorem ent_of_basicOpen_subset {u w : Finset a} (hw : w  in  A.Con)
    (hsub : A.basicOpen w  subseteq  A.basicOpen u) {a : a} (ha : a  in  u) : A.Ent w a := by
  have : A.closure w hw  in  A.basicOpen w := subset_closure A hw
  have hU : A.closure w hw  in  A.basicOpen u := hsub this
  exact hU (Finset.mem_coe.2 ha)

theorem con_of_basicOpen_subset {u w : Finset a} (hw : w  in  A.Con)
    (hsub : A.basicOpen w  subseteq  A.basicOpen u) : u  in  A.Con := by
  have hEnt : A.EntSet w u := fun a ha => ent_of_basicOpen_subset A hw hsub ha
  exact A.con_subset (proposition_2_3_ii A hw hEnt) (subset_funion_right _ _)

/-- **`infoSys_to_neighborhoodSystem`.** -/
def toNeighborhoodSystem : NeighborhoodSystem A.Element where
  mem X := exists  u, u  in  A.Con  /\  X = A.basicOpen u
  master := Set.univ
  master_mem := <{}, A.con_empty, (basicOpen_empty A).symm>
  inter_mem := by
    intro X Y Z hX hY hZ hZsub
    obtain <u, hu, rfl> := hX
    obtain <v, hv, rfl> := hY
    obtain <w, hw, rfl> := hZ
    have hsub : A.basicOpen w  subseteq  A.basicOpen (u  U ' v) := by
      intro x hx
      have hx' : x  in  A.basicOpen u  I  A.basicOpen v := hZsub hx
      rwa [basicOpen_inter A] at hx'
    have huv : u  U ' v  in  A.Con := con_of_basicOpen_subset A hw hsub
    exact <u  U ' v, huv, basicOpen_inter A u v>
  sub_master := fun {_} _ => Set.subset_univ _

theorem mem_basicOpen_of_singletons (f : (toNeighborhoodSystem A).Element) {Y : Finset a}
    (hY : forall  a  in  Y, f.mem (A.basicOpen ({a} : Finset a))) :
    f.mem (A.basicOpen Y) := by
  induction Y using Finset.induction with
  | empty =>
    simpa [basicOpen_empty] using f.master_mem
  | insert a s _ha ih =>
    have hA := hY a (Finset.mem_insert_self a s)
```

```
    have hSm := ih fun i hi => hY i (Finset.mem_insert_of_mem hi)
    simpa [basicOpen_singleton_inter A a s] using f.inter_mem hA hSm

def toFilter (x : A.Element) : (toNeighborhoodSystem A).Element where
  mem U := exists  u, u  in  A.Con  /\  U = A.basicOpen u  /\  u  subseteq  x.carrier
  sub := by
    intro U h
    obtain <u, hu, rfl, _> := h
    exact <u, hu, rfl>
  master_mem := <{}, A.con_empty, (basicOpen_empty A).symm, by
    intro a ha
    exact False.elim (Finset.notMem_empty a (Finset.mem_coe.1 ha))>
  inter_mem := by
    intro U V hU hV
    obtain <u, hu, rfl, huX> := hU
    obtain <v, hv, rfl, hvX> := hV
    refine <u  U ' v, ?_, basicOpen_inter A u v, ?_>
    * have hsub : (u  U ' v)  subseteq  x.carrier := by
        intro a ha
        rcases mem_funion.mp (Finset.mem_coe.1 ha) with h | h
        * exact huX (Finset.mem_coe.2 h)
        * exact hvX (Finset.mem_coe.2 h)
      exact x.consistent (u  U ' v) hsub
    * intro a ha
      rcases mem_funion.mp (Finset.mem_coe.1 ha) with h | h
      * exact huX (Finset.mem_coe.2 h)
      * exact hvX (Finset.mem_coe.2 h)
  up_mem := by
    intro U V hU hV hUV
    obtain <u, hu, rfl, huX> := hU
    obtain <v, hv, rfl> := hV
    refine <v, hv, rfl, ?_>
    intro a ha
    exact x.closed u a huX (ent_of_basicOpen_subset A hu hUV ha)

def ofFilter (f : (toNeighborhoodSystem A).Element) : A.Element where
  carrier := {a | f.mem (A.basicOpen ({a} : Finset a))}
  consistent := by
    intro Y hY
    have hmem : f.mem (A.basicOpen Y) :=
      mem_basicOpen_of_singletons A f fun a ha => hY (Finset.mem_coe.2 ha)
    obtain <u, hu, heq> := f.sub hmem
    have hsub : A.basicOpen u  subseteq  A.basicOpen Y := by
      intro x hx; exact heq  |>  hx
    exact con_of_basicOpen_subset A hu hsub
  closed := by
    intro Y b hY hEnt
    have hmemY : f.mem (A.basicOpen Y) :=
      mem_basicOpen_of_singletons A f fun c hc => hY (Finset.mem_coe.2 hc)
    have hsub : A.basicOpen Y  subseteq  A.basicOpen ({b} : Finset a) := by
      intro x hx c hc
      have eqcb : c = b := Finset.mem_singleton.mp (Finset.mem_coe.1 hc)
      rw [eqcb]
      exact x.closed Y b hx hEnt
    exact f.up_mem hmemY <({b} : Finset a), A.con_sing b, rfl> hsub

theorem mem_basicOpen_iff (f : (toNeighborhoodSystem A).Element) {u : Finset a}
    (_hu : u  in  A.Con) :
    f.mem (A.basicOpen u)  <->  u  subseteq  (ofFilter A f).carrier := by
  constructor
  * intro h a ha
    have hsub : A.basicOpen u  subseteq  A.basicOpen ({a} : Finset a) := by
      intro x hx c hc
```

```
      have hc' : c = a := Finset.mem_singleton.mp (Finset.mem_coe.1 hc)
      subst hc'
      exact hx (Finset.mem_coe.2 ha)
    exact f.up_mem h <({a} : Finset a), A.con_sing a, rfl> hsub
  * intro hsub
    exact mem_basicOpen_of_singletons A f fun a ha => hsub (Finset.mem_coe.2 ha)

theorem toFilter_ofFilter (f : (toNeighborhoodSystem A).Element) :
    toFilter A (ofFilter A f) = f := by
  refine NeighborhoodSystem.Element.ext (V := toNeighborhoodSystem A) fun U => ?_
  constructor
  * intro h
    obtain <u, hu, hU, hsub> := h
    subst hU
    exact (mem_basicOpen_iff A f hu).mpr hsub
  * intro hU
    obtain <u, hu, heq> := f.sub hU
    subst heq
    exact <u, hu, rfl, (mem_basicOpen_iff A f hu).mp hU>

theorem ofFilter_toFilter (x : A.Element) : ofFilter A (toFilter A x) = x := by
  have hc : (ofFilter A (toFilter A x)).carrier = x.carrier := by
    ext a
    constructor
    * intro ha
      obtain <u, hu, heq, hsub> := ha
      have huA : A.closure u hu  in  A.basicOpen u := subset_closure A hu
      have hsing : A.closure u hu  in  A.basicOpen ({a} : Finset a) := by
        rw [heq]; exact huA
      have ha' : a  in  (A.closure u hu).carrier := (mem_basicOpen_singleton A).1 hsing
      exact x.closed u a hsub ha'
    * intro ha
      refine <({a} : Finset a), A.con_sing a, rfl, ?_>
      intro c hc
      have hc' : c = a := Finset.mem_singleton.mp (Finset.mem_coe.1 hc)
      exact hc'  |>  ha
  rcases x with <c, cons, clo>
  rcases hte : ofFilter A (toFilter A <c, cons, clo>) with <c', cons', clo'>
  have hc' : c' = c := by simpa [hte] using hc
  subst hc'
  rfl

def domainOrderIso : A.Element ?o (toNeighborhoodSystem A).Element where
  toFun := toFilter A
  invFun := ofFilter A
  left_inv := ofFilter_toFilter A
  right_inv := toFilter_ofFilter A
  map_rel_iff' := by
    intro x y
    constructor
    * intro h a ha
      have hx : (toFilter A x).mem (A.basicOpen ({a} : Finset a)) :=
        <({a} : Finset a), A.con_sing a, rfl, by
          intro c hc
          have hc' : c = a := Finset.mem_singleton.mp (Finset.mem_coe.1 hc)
          exact hc'  |>  ha>
      have hy : (toFilter A y).mem (A.basicOpen ({a} : Finset a)) := h _ hx
      obtain <u, hu, heq, hsub> := hy
      have huA : A.closure u hu  in  A.basicOpen u := subset_closure A hu
      have hsing : A.closure u hu  in  A.basicOpen ({a} : Finset a) := by
        rw [heq]; exact huA
      have ha' : a  in  (A.closure u hu).carrier := (mem_basicOpen_singleton A).1 hsing
      exact y.closed u a hsub ha'
```

```
    * intro h U hU
      obtain <u, hu, hUeq, huX> := hU
      subst hUeq
      exact <u, hu, rfl, fun a ha => h (huX ha)>

end InfoSysToNeighborhood

/-- Blueprint-facing name for the 1982 -> 1980 basic-open neighbourhood system. -/
abbrev infoSys_to_neighborhoodSystem {a : Type*} [DecidableEq a] (A : InfoSys a) :
    NeighborhoodSystem A.Element :=
  InfoSysToNeighborhood.toNeighborhoodSystem A

end ScottModels
```

A.4 ScottModels/ContinuousLatticeToNeighborhood.lean

227 lines.

Lean 4 source

```
/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/scott_models
-/

import Mathlib.Order.Hom.Basic
import Scott1972.ContinuousLattice.WayBelow
import Scott1980.Neighborhood.Basic

/-!
# Continuous lattices -> neighbourhood systems (1972 -> 1980)

For a continuous lattice `D`, Scott?s way-below neighbourhoods
` Up a = { z | a << z }` form a `NeighborhoodSystem` on token set `D`.

Arbitrary filters need not be principal (`{ Up a | a <= x }` is a filter with
`ofFilter = x` but properly contains `toFilter x`). The correct domain is the
**round** filters: ` Up a in f` implies `exists b, a << b /\ Up b in f`. Under
`IsContinuousLattice`, `D ?o` round filters via `toFilter` / `ofFilter`.
-/

namespace ScottModels

open Scott1972.ContinuousLattice
open Scott1980.Neighborhood
open scoped Scott1972.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

/-- ` Up a = { z | a << z }`. -/
def wayBelowUp (a : D) : Set D := {z | a << z}

theorem wayBelowUp_bot : wayBelowUp (False : D) = (Set.univ : Set D) := by
  ext z
  exact <fun _ => trivial, fun _ => bot_wayBelow z>

theorem wayBelowUp_inter (a b : D) :
    wayBelowUp a ∩ wayBelowUp b = wayBelowUp (a ⊔ b) := by
```

```
  ext z
  constructor
  * exact fun <ha, hb> => WayBelow.sup ha hb
  * intro h
    exact <WayBelow.le_trans le_sup_left h, WayBelow.le_trans le_sup_right h>

theorem wayBelowUp_anti {a b : D} (hab : a  <=  b) :
    wayBelowUp b  subseteq  wayBelowUp a :=
  fun _ h => WayBelow.le_trans hab h

/-- Neighbourhood system of all ` Up a`. -/
def toNeighborhoodSystem : NeighborhoodSystem D where
  mem X := exists  a : D, X = wayBelowUp a
  master := Set.univ
  master_mem := <False, wayBelowUp_bot.symm>
  inter_mem := by
    intro X Y _Z hX hY _hZ _hZsub
    obtain <a, rfl> := hX
    obtain <b, rfl> := hY
    exact <a  sqsup  b, wayBelowUp_inter a b>
  sub_master := fun {_} _ => Set.subset_univ _

/-- Filter of the ` Up `-neighbourhood system on `D`. -/
abbrev Filter : Type _ :=
  (toNeighborhoodSystem : NeighborhoodSystem D).Element

/-- Principal filter of ` Up `-neighbourhoods at `x`. -/
def toFilter (x : D) : Filter (D := D) where
  mem U := exists  a : D, U = wayBelowUp a  /\  a  <<  x
  sub := by
    intro U h
    obtain <a, <rfl, _>> := h
    exact <a, rfl>
  master_mem := <False, wayBelowUp_bot.symm, bot_wayBelow x>
  inter_mem := by
    intro U V hU hV
    obtain <a, <rfl, ha>> := hU
    obtain <b, <rfl, hb>> := hV
    exact <a  sqsup  b, <wayBelowUp_inter a b, WayBelow.sup ha hb>>
  up_mem := by
    intro U V hU hV hUV
    obtain <a, <rfl, ha>> := hU
    obtain <b, rfl> := hV
    exact <b, <rfl, hUV ha>>

theorem toFilter_mono {x y : D} (hxy : x  <=  y) : toFilter x  <=  toFilter y := by
  intro U hU
  obtain <a, <rfl, ha>> := hU
  exact <a, <rfl, ha.trans_le hxy>>

theorem mem_wayBelowUp_toFilter {x a : D} :
    (toFilter x).mem (wayBelowUp a)  <->  a  <<  x := by
  constructor
  * intro h
    obtain <b, <heq, hb>> := h
    have : x  in  wayBelowUp b := hb
    rwa [ <-  heq] at this
  * intro ha
    exact <a, <rfl, ha>>

/-- Roundness: ` Up a  in  f` is witnessed by a finer code `b` with `a  <<  b`. -/
def IsRound (f : Filter (D := D)) : Prop :=
  forall  {a : D}, f.mem (wayBelowUp a)  ->  exists  b : D, a  <<  b  /\  f.mem (wayBelowUp b)
```

```
/-- Codes whose ` Up `-neighbourhoods lie in the filter. -/
def codes (f : Filter (D := D)) : Set D :=
  {a : D | f.mem (wayBelowUp a)}

theorem mem_codes_iff {f : Filter (D := D)} {a : D} :
    a  in  codes f  <->  f.mem (wayBelowUp a) :=
  Iff.rfl

theorem bot_mem_codes (f : Filter (D := D)) : (False : D)  in  codes f := by
  have : f.mem (wayBelowUp (False : D)) := by
    rw [wayBelowUp_bot]
    exact f.master_mem
  exact this

theorem codes_nonempty (f : Filter (D := D)) : (codes f).Nonempty :=
  <False, bot_mem_codes f>

theorem codes_directed (f : Filter (D := D)) : DirectedOn (*  <=  *) (codes f) := by
  intro a ha b hb
  refine <a  sqsup  b, ?_, le_sup_left, le_sup_right>
  have : f.mem (wayBelowUp a  I  wayBelowUp b) := f.inter_mem ha hb
  rwa [wayBelowUp_inter] at this

theorem codes_lower (f : Filter (D := D)) {a b : D} (hba : b  <=  a)
    (ha : a  in  codes f) : b  in  codes f :=
  f.up_mem ha <b, rfl> (wayBelowUp_anti hba)

/-- Retraction: ` sqsup ` of codes present in the filter. -/
noncomputable def ofFilter (f : Filter (D := D)) : D :=
  sSup (codes f)

theorem mem_wayBelowUp_ofFilter_of_round {f : Filter (D := D)} (hr : IsRound f) {a : D} :
    f.mem (wayBelowUp a)  <->  a  <<  ofFilter f := by
  constructor
  * intro ha
    obtain <b, hab, hb> := hr ha
    exact (wayBelow_sSup_iff (codes_nonempty f) (codes_directed f)).2 <b, hb, hab>
  * intro ha
    obtain <b, hbS, hab> := (wayBelow_sSup_iff (codes_nonempty f) (codes_directed f)).1 ha
    exact f.up_mem hbS <a, rfl> (wayBelowUp_anti hab.le)

theorem toFilter_ofFilter {f : Filter (D := D)} (hr : IsRound f) :
    toFilter (ofFilter f) = f := by
  refine NeighborhoodSystem.Element.ext (V := toNeighborhoodSystem) fun U => ?_
  constructor
  * intro hU
    obtain <a, <rfl, ha>> := hU
    exact (mem_wayBelowUp_ofFilter_of_round hr).2 ha
  * intro hU
    obtain <a, rfl> := f.sub hU
    refine <a, <rfl, ?_>>
    exact (mem_wayBelowUp_ofFilter_of_round hr).1 hU

section Continuous

variable (hD : IsContinuousLattice D)
include hD

theorem toFilter_isRound (x : D) : IsRound (toFilter x) := by
  intro a ha
  have ha' : a  <<  x := mem_wayBelowUp_toFilter.mp ha
  obtain <b, hab, hbx> := wayBelow_interpolate hD ha'
```

```
  exact <b, hab, mem_wayBelowUp_toFilter.mpr hbx>

theorem ofFilter_toFilter (x : D) : ofFilter (toFilter x) = x := by
  have hset : codes (toFilter x) = {a : D | a  <<  x} := by
    ext a
    exact mem_wayBelowUp_toFilter
  change sSup (codes (toFilter x)) = x
  rw [hset, hD.sSup_wayBelow x]

/-- Round filters of the ` Up `-system. -/
abbrev RoundFilter : Type _ :=
  { f : Filter (D := D) // IsRound f }

/-- Order-isomorphism: continuous lattice  <->  round ` Up `-filters. -/
noncomputable def domainOrderIso : D ?o RoundFilter (D := D) where
  toFun x := <toFilter x, toFilter_isRound hD x>
  invFun f := ofFilter (D := D) f.1
  left_inv x := ofFilter_toFilter hD x
  right_inv f := Subtype.ext (toFilter_ofFilter f.2)
  map_rel_iff' := by
    intro x y
    constructor
    * intro h
      have : ofFilter (toFilter x)  <=  ofFilter (toFilter y) := by
        refine sSup_le fun a ha => ?_
        have ha' : a  <<  x := mem_wayBelowUp_toFilter.mp ha
        exact le_sSup (h (wayBelowUp a) <a, <rfl, ha'>>)
      simpa [ofFilter_toFilter hD] using this
    * intro hxy
      exact toFilter_mono hxy

/-- Order-embedding into all filters (forgets roundness). -/
noncomputable def domainEmbedding :
    D ?o Filter (D := D) where
  toFun := toFilter
  inj' := by
    intro x y h
    rw [ <-  ofFilter_toFilter hD x,  <-  ofFilter_toFilter hD y, h]
  map_rel_iff' := by
    intro x y
    constructor
    * intro h
      have : ofFilter (toFilter x)  <=  ofFilter (toFilter y) := by
        refine sSup_le fun a ha => ?_
        have ha' : a  <<  x := mem_wayBelowUp_toFilter.mp ha
        exact le_sSup (h (wayBelowUp a) <a, <rfl, ha'>>)
      simpa [ofFilter_toFilter hD] using this
    * exact fun hxy => toFilter_mono hxy

end Continuous

end ContinuousLatticeToNeighborhood

/-- Blueprint name. -/
abbrev continuousLattice_to_neighborhoodSystem {D : Type*} [CompleteLattice D]
    (_hD : IsContinuousLattice D) : NeighborhoodSystem D :=
  ContinuousLatticeToNeighborhood.toNeighborhoodSystem

end ScottModels
```

A.5 ScottModels/InfoSysToIdealCompletion.lean

183 lines.

Lean 4 source

```
/-
Copyright (c) 2026  Lars Warren Ericson.  All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github:  https://github.com/catskillsresearch/scott_models
-/

import Mathlib.Order.Hom.Basic
import Mathlib.Order.Ideal
import Scott1982.Factoid45

/-!
# Information systems  ->  ideal completion (1982  ->  algebraic dcpo)

The domain `|A|` of an information system is the **ideal completion** of its
poset of finite elements `u` (closures of consistent finite token sets):
`A.Element ?o Order.Ideal (FiniteElement A)`.

Packaging of Factoids 4.4-4.5 (`directedSup`, algebraicity, `compact_closure`).
-/

namespace ScottModels

open Scott1982
open Scott1982.Constructive
open Order

namespace InfoSysToIdealCompletion

variable {a : Type*} [DecidableEq a] (A : InfoSys a)

/-- Finite (compact) elements: closures of consistent finite token sets. -/
abbrev FiniteElement : Type _ :=
  { x : A.Element // exists  (u : Finset a) (hu : u  in  A.Con), x = A.closure u hu }

theorem isFinite_bot :
    exists  (u : Finset a) (hu : u  in  A.Con), A.botElement = A.closure u hu :=
  <{}, A.con_empty, A.botElement_eq_closure_empty>

/-- Bottom, as a finite element. -/
def botFinite : FiniteElement A := <A.botElement, isFinite_bot A>

/-- Finite approximants of `x`, as `FiniteElement`s. -/
def finiteApproximants (x : A.Element) : Set (FiniteElement A) :=
  { y | (y : A.Element)  in  A.finiteApproximants x }

theorem mem_finiteApproximants_of_le {x : A.Element} {y : FiniteElement A}
    (hle : (y : A.Element)  <=  x) : y  in  finiteApproximants A x := by
  rcases y with <yval, <u, hu, rfl>>
  refine <u, hu, ?_, rfl>
  intro a ha
  exact hle (A.subset_closure hu ha)

theorem le_of_mem_finiteApproximants {x : A.Element} {y : FiniteElement A}
    (hy : y  in  finiteApproximants A x) : (y : A.Element)  <=  x := by
  obtain <u, hu, huX, hyeq> := hy
  exact hyeq  |>  A.closure_le_element x hu huX
```

```
theorem mem_finiteApproximants_iff {x : A.Element} {y : FiniteElement A} :
    y  in  finiteApproximants A x  <->  (y : A.Element)  <=  x :=
  <le_of_mem_finiteApproximants A, mem_finiteApproximants_of_le A>

theorem nonempty_finiteApproximants (x : A.Element) :
    (finiteApproximants A x).Nonempty :=
  <botFinite A, mem_finiteApproximants_of_le A (A.botElement_le x)>

theorem directed_finiteApproximants (x : A.Element) :
    DirectedOn (*  <=  *) (finiteApproximants A x) := by
  intro y_1 hy_1 y_2 hy_2
  obtain <u, hu, huX, hy_1eq> := hy_1
  obtain <v, hv, hvX, hy_2eq> := hy_2
  obtain <w, hw, hwX, huw, hvw> := A.closures_directed x hu hv huX hvX
  refine <<A.closure w hw, <w, hw, rfl>>, <w, hw, hwX, rfl>, ?_, ?_>
  * -- y_1  <=  closure w
    change (y_1 : A.Element)  <=  A.closure w hw
    exact hy_1eq  |>  huw
  * change (y_2 : A.Element)  <=  A.closure w hw
    exact hy_2eq  |>  hvw

theorem isLowerSet_finiteApproximants (x : A.Element) :
    IsLowerSet (finiteApproximants A x) := by
  intro y_1 y_2 hle hy_2
  exact mem_finiteApproximants_of_le A (le_trans hle (le_of_mem_finiteApproximants A hy_2))

/-- Ideal of finite approximants of `x`. -/
def toIdeal (x : A.Element) : Ideal (FiniteElement A) where
  carrier := finiteApproximants A x
  lower' := isLowerSet_finiteApproximants A x
  nonempty' := nonempty_finiteApproximants A x
  directed' := directed_finiteApproximants A x

theorem mem_toIdeal_iff {x : A.Element} {y : FiniteElement A} :
    y  in  toIdeal A x  <->  (y : A.Element)  <=  x :=
  mem_finiteApproximants_iff A

/-- Underlying set of elements of an ideal of finite elements. -/
def idealCarrier (I : Ideal (FiniteElement A)) : Set A.Element :=
  Subtype.val '' (I : Set (FiniteElement A))

theorem nonempty_idealCarrier (I : Ideal (FiniteElement A)) :
    (idealCarrier A I).Nonempty := by
  obtain <y, hy> := I.nonempty
  exact <y, y, hy, rfl>

theorem directed_idealCarrier (I : Ideal (FiniteElement A)) :
    A.IsDirected (idealCarrier A I) := by
  intro x y hx hy
  obtain <x', hx', rfl> := hx
  obtain <y', hy', rfl> := hy
  obtain <z', hz', hxz, hyz> := I.directed x' hx' y' hy'
  exact <z', <z', hz', rfl>, hxz, hyz>

/-- Retraction: directed supremum of the finite elements in the ideal. -/
noncomputable def ofIdeal (I : Ideal (FiniteElement A)) : A.Element :=
  A.directedSup (idealCarrier A I) (nonempty_idealCarrier A I) (directed_idealCarrier A I)

theorem le_ofIdeal_of_mem {I : Ideal (FiniteElement A)} {y : FiniteElement A}
    (hy : y  in  I) : (y : A.Element)  <=  ofIdeal A I :=
  A.le_directedSup _ _ _ <y, hy, rfl>

theorem ofIdeal_toIdeal (x : A.Element) : ofIdeal A (toIdeal A x) = x := by
```

```
  -- idealCarrier (toIdeal x) = finiteApproximants as Elements = A.finiteApproximants x
  have hset : idealCarrier A (toIdeal A x) = A.finiteApproximants x := by
    ext z
    constructor
    * intro hz
      obtain <y, hy, rfl> := hz
      exact hy
    * intro hz
      refine <<z, ?_>, hz, rfl>
      obtain <u, hu, _, rfl> := hz
      exact <u, hu, rfl>
  -- directedSup of that set is x
  change A.directedSup (idealCarrier A (toIdeal A x)) _ _ = x
  simp_rw [hset]
  exact (A.eq_directedSup_finiteApproximants x).symm

theorem toIdeal_ofIdeal (I : Ideal (FiniteElement A)) : toIdeal A (ofIdeal A I) = I := by
  refine Ideal.ext ?_
  ext y
  constructor
  * intro hy
    have hle : (y : A.Element)  <=  ofIdeal A I := le_of_mem_finiteApproximants A hy
    rcases y with <yval, <u, hu, rfl>>
    obtain <z, hz, hcle> :=
      A.compact_closure (idealCarrier A I) (nonempty_idealCarrier A I)
        (directed_idealCarrier A I) hu hle
    obtain <z', hz', rfl> := hz
    exact I.lower hcle hz'
  * intro hy
    exact mem_finiteApproximants_of_le A (le_ofIdeal_of_mem A hy)

theorem toIdeal_mono {x y : A.Element} (hxy : x  <=  y) : toIdeal A x  <=  toIdeal A y := by
  intro z hz
  exact mem_finiteApproximants_of_le A
    (le_trans (le_of_mem_finiteApproximants A hz) hxy)

/-- Order isomorphism: domain elements  <->  ideals of finite elements. -/
noncomputable def domainOrderIso : A.Element ?o Ideal (FiniteElement A) where
  toFun := toIdeal A
  invFun := ofIdeal A
  left_inv := ofIdeal_toIdeal A
  right_inv := toIdeal_ofIdeal A
  map_rel_iff' := by
    intro x y
    constructor
    * intro h
      -- x  <=  y from toIdeal x  <=  toIdeal y via algebraicity
      rw [ <-  ofIdeal_toIdeal A x,  <-  ofIdeal_toIdeal A y]
      refine A.directedSup_le _ _ _ ?_
      intro z hz
      obtain <z', hz', rfl> := hz
      exact le_ofIdeal_of_mem A (h hz')
    * exact toIdeal_mono A

end InfoSysToIdealCompletion

/-- Blueprint name: `|A|` as the ideal completion of its finite elements. -/
noncomputable abbrev infoSys_to_idealCompletion {a : Type*} [DecidableEq a] (A : InfoSys a) :
    A.Element ?o Ideal (InfoSysToIdealCompletion.FiniteElement A) :=
  InfoSysToIdealCompletion.domainOrderIso A

end ScottModels
```

A.6 ScottModels/IdealCompletionToContinuousLattice.lean

78 lines.

Lean 4 source

```
/-
Copyright (c) 2026  Lars Warren Ericson.  All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github:  https://github.com/catskillsresearch/scott_models
-/

import Mathlib.Order.CompleteLattice.Basic
import Mathlib.Order.Directed
import Scott1972.ContinuousLattice.WayBelow

/-!
# Algebraic complete lattices  ->  continuous lattices (ideal-completion  ->  1972)

An **algebraic** complete lattice -- every element is the directed supremum of the
compact elements below it -- is a continuous lattice in Scott's sense
(`IsContinuousLattice`). Compact elements are way-below anything they lie under,
via Scott-openness of `Set.Ici k`.

This is the classical frontier of this article?s blueprint
(`idealCompletion_to_continuousLattice`): Scott's ` << ` is topological.
-/

namespace ScottModels

open Scott1972.ContinuousLattice
open scoped Scott1972.ContinuousLattice

namespace IdealCompletionToContinuousLattice

variable {D : Type*} [CompleteLattice D]

/-- Order-theoretic compactness: inaccessible by nonempty directed suprema. -/
def IsCompactElement (k : D) : Prop :=
  forall  ?S : Set D?, S.Nonempty  ->  DirectedOn (*  <=  *) S  ->  k  <=  sSup S  ->  exists
    s  in  S, k  <=  s

/-- Compactness implies `Set.Ici k` is Scott-open. -/
theorem scottOpen_Ici_of_compact {k : D} (hk : IsCompactElement k) :
    ScottOpen (Set.Ici k) := by
  refine <isUpperSet_Ici k, fun S hS hSdir hmem => ?_>
  obtain <s, hsS, hks> := hk hS hSdir (Set.mem_Ici.1 hmem)
  exact <s, hsS, Set.mem_Ici.2 hks>

/-- Compact elements are way below anything above them. -/
theorem compact_wayBelow {k y : D} (hk : IsCompactElement k) (hky : k  <=  y) : k  <<  y :=
  <Set.Ici k, scottOpen_Ici_of_compact hk, Set.mem_Ici.2 hky, subset_rfl>

/-- Algebraicity: every element is the directed lub of compact elements below it. -/
def IsAlgebraicLattice (D : Type*) [CompleteLattice D] : Prop :=
  forall  y : D,
    let S := {x : D | IsCompactElement x  /\  x  <=  y}
    S.Nonempty  /\  DirectedOn (*  <=  *) S  /\  sSup S = y

/-- **Blueprint:** algebraic complete lattice  =>  continuous lattice (Scott Def. 2.3). -/
theorem isContinuousLattice_of_algebraic (hA : IsAlgebraicLattice D) :
    IsContinuousLattice D := by
  intro y
```

```
  obtain <hne, hdir, hsup> := hA y
  refine <?_, fun z hz => ?_>
  * -- y is an upper bound of `{x | x  <<  y}`
    intro x hx
    exact hx.le
  * -- any upper bound of the way-belows is  >=  y
    -- every compact  <=  y is  <<  y, hence  <=  z
    have hle : sSup {x : D | IsCompactElement x  /\  x  <=  y}  <=  z := by
      refine sSup_le fun x hx => ?_
      exact hz (compact_wayBelow hx.1 hx.2)
    exact hsup  |>  hle

end IdealCompletionToContinuousLattice

/-- Blueprint name. -/
abbrev idealCompletion_to_continuousLattice {D : Type*} [CompleteLattice D]
    (hA : IdealCompletionToContinuousLattice.IsAlgebraicLattice D) :
    IsContinuousLattice D :=
  IdealCompletionToContinuousLattice.isContinuousLattice_of_algebraic hA

end ScottModels
```

A.7 ScottModels/PresentationDomains.lean

168 lines.

Lean 4 source

```
/-
Copyright (c) 2026  Lars Warren Ericson.  All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github:  https://github.com/catskillsresearch/scott_models
-/

import Mathlib.Order.Hom.Basic
import ScottModels.InfoSysToNeighborhood
import ScottModels.InfoSysToIdealCompletion
import ScottModels.NeighborhoodToInfoSys
import ScottModels.ContinuousLatticeToNeighborhood
import ScottModels.IdealCompletionToContinuousLattice

/-!
# Presentation domain equivalences

## InfoSys / neighbourhood / ideal (constructive)

For any information system `A`:
`|A| ?o` basic-open neighbourhood filters ?o `Ideal (FiniteElement A)`.

Under a decidable `NbhdBasis`, the same triangle starts from `|?|`.

## Continuous lattices (1972 corner)

Points of a continuous lattice `D` are **round** ` Up `-filters, not arbitrary filters:
`D ?o RoundFilter`. With `DecidableEq D`, the ` Up `-system admits an `NbhdBasis`,
so round filters transport to a subtype of the induced InfoSys domain (and thence
into the ideal-completion triangle). The full (non-round) `|?|` / `|A|` is properly
larger.
-/

namespace ScottModels
```

```
open Scott1982
open Scott1972.ContinuousLattice
open Scott1980.Neighborhood
open Order
open scoped Scott1972.ContinuousLattice
open ContinuousLatticeToNeighborhood

variable {a : Type*} [DecidableEq a]

/-- Neighbourhood filters of `|A|`  <->  ideals of finite elements (via `|A|`). -/
noncomputable def neighborhood_ideal_iso (A : InfoSys a) :
    (InfoSysToNeighborhood.toNeighborhoodSystem A).Element ?o
      Ideal (InfoSysToIdealCompletion.FiniteElement A) :=
  (InfoSysToNeighborhood.domainOrderIso A).symm.trans
    (InfoSysToIdealCompletion.domainOrderIso A)

variable {? b : Type*} [DecidableEq ?]

/-- Under a decidable neighbourhood basis, `|?|`  <->  InfoSys domain  <->  ideal completion.
    -/
noncomputable def nbhdBasis_ideal_iso (B : NbhdBasis ? b) :
    B.system.Element ?o Ideal (InfoSysToIdealCompletion.FiniteElement B.toInfoSys) :=
  B.domainOrderIso.trans (InfoSysToIdealCompletion.domainOrderIso B.toInfoSys)

/-- Constructive 1980 <-> 1982 <-> ideal triangle for InfoSys domains. -/
noncomputable abbrev presentation_domains_equiv_infoSys (A : InfoSys a) :=
  neighborhood_ideal_iso A

/-! ## Continuous lattice presentations via round ` Up `-filters -/

variable {D : Type*} [CompleteLattice D] [DecidableEq D]

/-- Decidable coding of the ` Up `-neighbourhood system: tokens are elements of `D`. -/
def wayBelowNbhdBasis : NbhdBasis D D where
  system := toNeighborhoodSystem
  nbhd := wayBelowUp
  nbhd_mem := fun a => <a, rfl>
  exhaustive := by
    intro X hX
    obtain <a, rfl> := hX
    exact <a, rfl>
  botIdx := False
  botIdx_eq := wayBelowUp_bot

theorem wayBelowNbhdBasis_system :
    (wayBelowNbhdBasis (D := D)).system = toNeighborhoodSystem :=
  rfl

/-- Roundness transported to InfoSys elements of the ` Up `-basis. -/
def IsRoundInfoSysElement (e : (wayBelowNbhdBasis (D := D)).toInfoSys.Element) : Prop :=
  IsRound ((wayBelowNbhdBasis (D := D)).domainOrderIso.symm e)

abbrev RoundInfoSysElement : Type _ :=
  { e : (wayBelowNbhdBasis (D := D)).toInfoSys.Element // IsRoundInfoSysElement (D := D) e }

/-- Round ` Up `-filters  <->  round elements of the coded InfoSys. -/
noncomputable def roundFilter_infoSys_iso :
    RoundFilter (D := D) ?o RoundInfoSysElement (D := D) where
  toFun := fun f =>
    <(wayBelowNbhdBasis (D := D)).domainOrderIso f.1, by
      change IsRound ((wayBelowNbhdBasis (D := D)).domainOrderIso.symm
        ((wayBelowNbhdBasis (D := D)).domainOrderIso f.1))
```

```
      rw [OrderIso.symm_apply_apply]
      exact f.2>
  invFun := fun e =>
    <(wayBelowNbhdBasis (D := D)).domainOrderIso.symm e.1, e.2>
  left_inv := fun f => Subtype.ext <|
    (wayBelowNbhdBasis (D := D)).domainOrderIso.left_inv f.1
  right_inv := fun e => Subtype.ext <|
    (wayBelowNbhdBasis (D := D)).domainOrderIso.right_inv e.1
  map_rel_iff' := by
    intro f g
    exact (wayBelowNbhdBasis (D := D)).domainOrderIso.map_rel_iff

section Continuous

variable (hD : IsContinuousLattice D)
include hD

/-- **1972 <-> round 1980:** continuous lattice <-> round ` Up `-filters. -/
noncomputable def continuousLattice_roundFilter_iso :
    D ?o RoundFilter (D := D) :=
  ContinuousLatticeToNeighborhood.domainOrderIso hD

/-- **1972 <-> round 1982:** continuous lattice <-> round InfoSys elements of the ` Up `-
    basis. -/
noncomputable def continuousLattice_roundInfoSys_iso :
    D ?o RoundInfoSysElement (D := D) :=
  (continuousLattice_roundFilter_iso hD).trans roundFilter_infoSys_iso

/-- Round InfoSys elements <-> ideals of finite elements that come from round InfoSys
elements (via the ideal-completion iso). -/
noncomputable def roundInfoSys_ideal_iso :
    RoundInfoSysElement (D := D) ?o
      { I : Ideal (InfoSysToIdealCompletion.FiniteElement
          (wayBelowNbhdBasis (D := D)).toInfoSys) //
        IsRoundInfoSysElement (D := D)
          ((InfoSysToIdealCompletion.domainOrderIso
            (wayBelowNbhdBasis (D := D)).toInfoSys).symm I) } := by
  let A := (wayBelowNbhdBasis (D := D)).toInfoSys
  let ?E := InfoSysToIdealCompletion.domainOrderIso A
  refine {
    toFun := fun e => <?E e.1, by
      change IsRoundInfoSysElement (?E.symm (?E e.1))
      rw [OrderIso.symm_apply_apply]
      exact e.2>
    invFun := fun I => <?E.symm I.1, I.2>
    left_inv := fun e => Subtype.ext (?E.left_inv e.1)
    right_inv := fun I => Subtype.ext (?E.right_inv I.1)
    map_rel_iff' := by
      intro e_1 e_2
      exact ?E.map_rel_iff
  }

/-- **Blueprint:** three-presentation equivalence for continuous lattices.

`D ?o RoundFilter ?o RoundInfoSysElement`, with the ` Up `-system?s `NbhdBasis`
supplying the 1980 <-> 1982 coding. (Raw `|?|` / full `|A|` remain larger.) -/
noncomputable def presentation_domains_equiv :
    D ?o RoundInfoSysElement (D := D) :=
  continuousLattice_roundInfoSys_iso hD

/-- Extended form through ideal completion of finite elements of the ` Up `-InfoSys. -/
noncomputable def presentation_domains_equiv_ideal :
    D ?o
```

```
      { I : Ideal (InfoSysToIdealCompletion.FiniteElement
          (wayBelowNbhdBasis (D := D)).toInfoSys) //
        IsRoundInfoSysElement (D := D)
          ((InfoSysToIdealCompletion.domainOrderIso
            (wayBelowNbhdBasis (D := D)).toInfoSys).symm I) } :=
  (presentation_domains_equiv hD).trans (roundInfoSys_ideal_iso (D := D))

end Continuous

end ScottModels
```

A.8 ScottModels/InfoSysConstructions.lean

469 lines.

Lean 4 source (lines 1–400)

```
/-
Copyright (c) 2026  Lars Warren Ericson.  All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github:  https://github.com/catskillsresearch/scott_models
-/

import Mathlib.Order.Hom.Basic
import Mathlib.Data.Sum.Order
import Mathlib.Order.WithBot
import Scott1982.Theorem72
import Scott1982.Proposition64

/-!
# Construction equivalence -- products, separated sums, and function spaces

* **Product:** `|A  x  B| ?o |A|  x  |B|` via `pairElements` / projections (Prop 6.2).
* **Separated sum:** `|A + B| ?o WithBot (|A| ? |B|)` via `inl`/`inr` (Prop 6.4).
  Classification of sum elements uses classical case-split on token polarity
  (`Classical.choice` in the footprint).
* **Function space:** `|A  ->  B| ?o ApproximableMap A B` via Theorem 7.2
  `approxMap_toElement` / `element_toApproxMap`.

1972 counterpart: products of continuous lattices (`proposition_2_9_a`); function space
Thm 3.3 in the sibling package.
-/

namespace ScottModels

open Scott1982
open Scott1982.Constructive
open Scott1982.InfoSys
open Scott1982.InfoSys.ApproximableMap

namespace InfoSysConstructions

variable {a b : Type*} [DecidableEq a] [DecidableEq b] (A : InfoSys a) (B : InfoSys b)

/-- Split a product-domain element into its two projections. -/
def unpair (z : (productSystem A B).Element) : A.Element  x  B.Element :=
  ((fstMap A B).toElement z, (sndMap A B).toElement z)

theorem unpair_pairElements (x : A.Element) (y : B.Element) :
    unpair A B (pairElements A B x y) = (x, y) := by
  simp [unpair, fstMap_pairElements, sndMap_pairElements]
```

```
theorem pairElements_unpair (z : (productSystem A B).Element) :
    pairElements A B ((fstMap A B).toElement z) ((sndMap A B).toElement z) = z :=
  element_eq_of_fst_snd A B _ z (fstMap_pairElements A B _ _) (sndMap_pairElements A B _ _)

theorem pairElements_mono {x_1 x_2 : A.Element} {y_1 y_2 : B.Element}
    (hx : x_1 ≤ x_2) (hy : y_1 ≤ y_2) :
    pairElements A B x_1 y_1 ≤ pairElements A B x_2 y_2 := by
  intro p hp
  exact <fun hbot => hx (hp.1 hbot), fun hbot => hy (hp.2 hbot)>

theorem unpair_mono {z_1 z_2 : (productSystem A B).Element} (hz : z_1 ≤ z_2) :
    unpair A B z_1 ≤ unpair A B z_2 := by
  constructor
  * exact (fstMap A B).toElement_mono hz
  * exact (sndMap A B).toElement_mono hz

/-- **Product domain iso (1982).** `|A × B| ?o |A| × |B|`. -/
noncomputable def productDomainIso :
    A.Element × B.Element ?o (productSystem A B).Element where
  toFun := fun p => pairElements A B p.1 p.2
  invFun := unpair A B
  left_inv := fun p => unpair_pairElements A B p.1 p.2
  right_inv := pairElements_unpair A B
  map_rel_iff' := by
    intro p q
    constructor
    * intro h
      simpa [unpair, fstMap_pairElements, sndMap_pairElements] using unpair_mono A B h
    * intro h
      exact pairElements_mono A B h.1 h.2

/-! ## Separated sum `|A + B| ?o WithBot (|A| ? |B|)` -/

private theorem rhtFinset_singleton_left (x : a) :
    rhtFinset ({SumToken.left x} : Finset (SumToken a b)) = {} := by
  ext y
  constructor
  * intro hy
    have : SumToken.right y ∈ ({SumToken.left x} : Finset _) := (mem_rhtFinset).1 hy
    exact False.elim (nomatch Finset.mem_singleton.mp this)
  * intro hy
    exact False.elim (Finset.notMem_empty y hy)

private theorem lftFinset_singleton_right (y : b) :
    lftFinset ({SumToken.right y} : Finset (SumToken a b)) = {} := by
  ext x
  constructor
  * intro hx
    have : SumToken.left x ∈ ({SumToken.right y} : Finset _) := (mem_lftFinset).1 hx
    exact False.elim (nomatch Finset.mem_singleton.mp this)
  * intro hx
    exact False.elim (Finset.notMem_empty x hx)

theorem left_bot_mem_inlMap_toElement (x : A.Element) :
    SumToken.left A.bot ∈ ((inlMap A B).toElement x).carrier := by
  refine <{A.bot}, ?_, ?_>
  * intro a ha
    have : a = A.bot := Finset.mem_singleton.mp (Finset.mem_coe.1 ha)
    subst this
    exact factoid_3_2 A x
  * refine <A.con_sing A.bot, ?_, rhtFinset_singleton_left (b := b) A.bot, ?_>
    * exact Or.inl <by rw [lftFinset_singleton_left]; exact A.con_sing A.bot,
        rhtFinset_singleton_left (b := b) A.bot>
```

```
    * rw [lftFinset_singleton_left]
      exact proposition_2_3_iii A (A.con_sing A.bot)

theorem right_bot_mem_inrMap_toElement (y : B.Element) :
    SumToken.right B.bot  in  ((inrMap A B).toElement y).carrier := by
  refine <{B.bot}, ?_, ?_>
  * intro b hb
    have : b = B.bot := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
    subst this
    exact factoid_3_2 B y
  * refine <B.con_sing B.bot, ?_, lftFinset_singleton_right (a := a) B.bot, ?_>
    * exact Or.inr <lftFinset_singleton_right (a := a) B.bot,
        by rw [rhtFinset_singleton_right]; exact B.con_sing B.bot>
    * rw [rhtFinset_singleton_right]
      exact proposition_2_3_iii B (B.con_sing B.bot)

theorem sumElementLft_inlMap_toElement (x : A.Element) :
    sumElementLft A B ((inlMap A B).toElement x) A.bot
      (left_bot_mem_inlMap_toElement A B x) = x := by
  apply le_antisymm
  * intro a ha
    -- ha : left a  in  inl(x)
    obtain <u, hu, <huCon, hSum, hr, hEnt>> := ha
    have hEntA : A.Ent u a := by
      have : lftFinset ({SumToken.left a} : Finset (SumToken a b)) = {a} :=
        lftFinset_singleton_left a
      simpa [this] using hEnt a (Finset.mem_singleton_self a)
    exact x.closed u a hu hEntA
  * intro a ha
    change SumToken.left a  in  ((inlMap A B).toElement x).carrier
    refine <{a}, ?_, ?_>
    * intro b hb
      have : b = a := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
      subst this; exact ha
    * refine <A.con_sing a, Or.inl <by rw [lftFinset_singleton_left]; exact A.con_sing a,
        rhtFinset_singleton_left (b := b) a>, rhtFinset_singleton_left (b := b) a, ?_>
      rw [lftFinset_singleton_left]
      exact proposition_2_3_iii A (A.con_sing a)

theorem sumElementRht_inrMap_toElement (y : B.Element) :
    sumElementRht A B ((inrMap A B).toElement y) B.bot
      (right_bot_mem_inrMap_toElement A B y) = y := by
  apply le_antisymm
  * intro b hb
    obtain <w, hw, <hwCon, hSum, hl, hEnt>> := hb
    have hEntB : B.Ent w b := by
      have : rhtFinset ({SumToken.right b} : Finset (SumToken a b)) = {b} :=
        rhtFinset_singleton_right b
      simpa [this] using hEnt b (Finset.mem_singleton_self b)
    exact y.closed w b hw hEntB
  * intro b hb
    change SumToken.right b  in  ((inrMap A B).toElement y).carrier
    refine <{b}, ?_, ?_>
    * intro c hc
      have : c = b := Finset.mem_singleton.mp (Finset.mem_coe.1 hc)
      subst this; exact hb
    * refine <B.con_sing b, Or.inr <lftFinset_singleton_right (a := a) b,
        by rw [rhtFinset_singleton_right]; exact B.con_sing b>,
        lftFinset_singleton_right (a := a) b, ?_>
      rw [rhtFinset_singleton_right]
      exact proposition_2_3_iii B (B.con_sing b)

theorem inlMap_toElement_injective :
```

```
    Function.Injective (inlMap A B).toElement := by
  intro x y h
  refine le_antisymm ?_ ?_
  * intro a ha
    have hx := sumElementLft_inlMap_toElement A B x
    have : SumToken.left a  in  ((inlMap A B).toElement x).carrier := by
      rw [ <-  hx] at ha; exact ha
    rw [h] at this
    have hy := sumElementLft_inlMap_toElement A B y
    have : a  in  (sumElementLft A B ((inlMap A B).toElement y) A.bot
        (left_bot_mem_inlMap_toElement A B y)).carrier := this
    rwa [hy] at this
  * intro a ha
    have hy := sumElementLft_inlMap_toElement A B y
    have : SumToken.left a  in  ((inlMap A B).toElement y).carrier := by
      rw [ <-  hy] at ha; exact ha
    rw [ <-  h] at this
    have hx := sumElementLft_inlMap_toElement A B x
    have : a  in  (sumElementLft A B ((inlMap A B).toElement x) A.bot
        (left_bot_mem_inlMap_toElement A B x)).carrier := this
    rwa [hx] at this

theorem inrMap_toElement_injective :
    Function.Injective (inrMap A B).toElement := by
  intro x y h
  refine le_antisymm ?_ ?_
  * intro b hb
    have hx := sumElementRht_inrMap_toElement A B x
    have : SumToken.right b  in  ((inrMap A B).toElement x).carrier := by
      rw [ <-  hx] at hb; exact hb
    rw [h] at this
    have hy := sumElementRht_inrMap_toElement A B y
    have : b  in  (sumElementRht A B ((inrMap A B).toElement y) B.bot
        (right_bot_mem_inrMap_toElement A B y)).carrier := this
    rwa [hy] at this
  * intro b hb
    have hy := sumElementRht_inrMap_toElement A B y
    have : SumToken.right b  in  ((inrMap A B).toElement y).carrier := by
      rw [ <-  hy] at hb; exact hb
    rw [ <-  h] at this
    have hx := sumElementRht_inrMap_toElement A B x
    have : b  in  (sumElementRht A B ((inrMap A B).toElement x) B.bot
        (right_bot_mem_inrMap_toElement A B x)).carrier := this
    rwa [hx] at this

/-- Every sum element is False, a pure left copy, or a pure right copy. Classical. -/
theorem sum_element_trichotomy (z : (sumSystem A B).Element) :
    z = (sumSystem A B).botElement  \/
      (exists  x : A.Element, z = (inlMap A B).toElement x)  \/
        (exists  y : B.Element, z = (inrMap A B).toElement y) := by
  classical
  by_cases hL : exists  x : a, SumToken.left x  in  z.carrier
  * obtain <x0, hx0> := hL
    exact Or.inr (Or.inl <sumElementLft A B z x0 hx0, (inlMap_toElement_sumElementLft A B z x0
    hx0).symm>)
  * by_cases hR : exists  y : b, SumToken.right y  in  z.carrier
    * obtain <y0, hy0> := hR
      exact Or.inr (Or.inr <sumElementRht A B z y0 hy0, (inrMap_toElement_sumElementRht A B z
    y0 hy0).symm>)
    * exact Or.inl (eq_botElement_of_no_injections A B z
        (fun x hx => hL <x, hx>) (fun y hy => hR <y, hy>))

/-- Classify a sum-domain element as `WithBot (|A| ? |B|)`. Classical. -/
```

```
noncomputable def classifySum (z : (sumSystem A B).Element) :
    WithBot (A.Element ? B.Element) := by
  classical
  exact if hL : exists  x : a, SumToken.left x  in  z.carrier then
    let x0 := Classical.choose hL
    some (.inl (sumElementLft A B z x0 (Classical.choose_spec hL)))
  else if hR : exists  y : b, SumToken.right y  in  z.carrier then
    let y0 := Classical.choose hR
    some (.inr (sumElementRht A B z y0 (Classical.choose_spec hR)))
  else
    False

/-- Assemble a sum-domain element from a separated-sum code. -/
def assembleSum : WithBot (A.Element ? B.Element)  ->  (sumSystem A B).Element
  | False => (sumSystem A B).botElement
  | some (.inl x) => (inlMap A B).toElement x
  | some (.inr y) => (inrMap A B).toElement y

theorem assembleSum_classifySum (z : (sumSystem A B).Element) :
    assembleSum A B (classifySum A B z) = z := by
  classical
  simp only [classifySum, assembleSum]
  split_ifs with hL hR
  * exact inlMap_toElement_sumElementLft A B z _ _
  * exact inrMap_toElement_sumElementRht A B z _ _
  * exact (eq_botElement_of_no_injections A B z
      (fun x hx => hL <x, hx>) (fun y hy => hR <y, hy>)).symm

theorem not_left_mem_sum_botElement {x : a} :
    SumToken.left x  notin  ((sumSystem A B).botElement).carrier := by
  intro hx
  have hEnt : (sumSystem A B).Ent {sumBot} (.left x) := hx
  -- Ent {bot} (left x) requires lftFinset {bot}  !=  {}
  rcases hEnt with <_, <hne, _>>
  exact hne lftFinset_singleton_bot

theorem not_right_mem_sum_botElement {y : b} :
    SumToken.right y  notin  ((sumSystem A B).botElement).carrier := by
  intro hy
  have hEnt : (sumSystem A B).Ent {sumBot} (.right y) := hy
  rcases hEnt with <_, <hne, _>>
  exact hne rhtFinset_singleton_bot

theorem sumElementLft_eq {z : (sumSystem A B).Element} {x0 x1 : a}
    (hx0 : SumToken.left x0  in  z.carrier) (hx1 : SumToken.left x1  in  z.carrier) :
    sumElementLft A B z x0 hx0 = sumElementLft A B z x1 hx1 := by
  refine le_antisymm ?_ ?_ <;> intro a ha <;> exact ha

theorem sumElementRht_eq {z : (sumSystem A B).Element} {y0 y1 : b}
    (hy0 : SumToken.right y0  in  z.carrier) (hy1 : SumToken.right y1  in  z.carrier) :
    sumElementRht A B z y0 hy0 = sumElementRht A B z y1 hy1 := by
  refine le_antisymm ?_ ?_ <;> intro b hb <;> exact hb

theorem classifySum_assembleSum (w : WithBot (A.Element ? B.Element)) :
    classifySum A B (assembleSum A B w) = w := by
  classical
  cases w with
  | bot =>
    simp only [assembleSum, classifySum]
    split_ifs with hL hR
    * exact False.elim (not_left_mem_sum_botElement A B (Classical.choose_spec hL))
    * exact False.elim (not_right_mem_sum_botElement A B (Classical.choose_spec hR))
    * rfl
```

```
  | coe s =>
    cases s with
    | inl x =>
      have hx : SumToken.left A.bot  in  ((inlMap A B).toElement x).carrier :=
        left_bot_mem_inlMap_toElement A B x
      simp only [assembleSum, classifySum]
      have hL : exists  a : a, SumToken.left a  in  ((inlMap A B).toElement x).carrier := <A.
    bot, hx>
      rw [dif_pos hL]
      congr 2
      -- choose_spec gives some witness; sumElementLft equals via bot witness
      exact (sumElementLft_eq A B (Classical.choose_spec hL) hx).trans
        (sumElementLft_inlMap_toElement A B x)
    | inr y =>
      have hy : SumToken.right B.bot  in  ((inrMap A B).toElement y).carrier :=
        right_bot_mem_inrMap_toElement A B y
      simp only [assembleSum, classifySum]
      have hR : exists  b : b, SumToken.right b  in  ((inrMap A B).toElement y).carrier := <B.
    bot, hy>
      -- Need not exists  left, so first if is false
      have hL : not exists  a : a, SumToken.left a  in  ((inrMap A B).toElement y).carrier :=
    by
        intro <a, ha>
        exact not_mem_right_of_mem_left A B _ ha hy
      rw [dif_neg hL, dif_pos hR]
      congr 2
      exact (sumElementRht_eq A B (Classical.choose_spec hR) hy).trans
        (sumElementRht_inrMap_toElement A B y)

theorem inlMap_toElement_le_iff {x y : A.Element} :
    (inlMap A B).toElement x  <=  (inlMap A B).toElement y  <->  x  <=  y := by
  constructor
  * intro h a ha
    have : SumToken.left a  in  ((inlMap A B).toElement x).carrier := by
      -- from sumElementLft_inl round-trip carrier
      have hx := sumElementLft_inlMap_toElement A B x
      -- a  in  x = sumElementLft  =>  left a  in  inl x
      have : a  in  (sumElementLft A B ((inlMap A B).toElement x) A.bot
          (left_bot_mem_inlMap_toElement A B x)).carrier := by
        simpa [hx] using ha
      exact this
    exact (sumElementLft_inlMap_toElement A B y)  |>
      (show a  in  (sumElementLft A B ((inlMap A B).toElement y) A.bot
          (left_bot_mem_inlMap_toElement A B y)).carrier from h this)
  * exact (inlMap A B).toElement_mono

theorem inrMap_toElement_le_iff {x y : B.Element} :
    (inrMap A B).toElement x  <=  (inrMap A B).toElement y  <->  x  <=  y := by
  constructor
  * intro h b hb
    have : SumToken.right b  in  ((inrMap A B).toElement x).carrier := by
      have hx := sumElementRht_inrMap_toElement A B x
      have : b  in  (sumElementRht A B ((inrMap A B).toElement x) B.bot
          (right_bot_mem_inrMap_toElement A B x)).carrier := by
        simpa [hx] using hb
      exact this
    exact (sumElementRht_inrMap_toElement A B y)  |>
      (show b  in  (sumElementRht A B ((inrMap A B).toElement y) B.bot
          (right_bot_mem_inrMap_toElement A B y)).carrier from h this)
  * exact (inrMap A B).toElement_mono

theorem assembleSum_mono {w_1 w_2 : WithBot (A.Element ? B.Element)} (h : w_1  <=  w_2) :
    assembleSum A B w_1  <=  assembleSum A B w_2 := by
```

```
  cases w_1 with
  | bot => exact botElement_le _ _
  | coe s_1 =>
    cases w_2 with
    | bot => exact (WithBot.not_coe_le_bot _ h).elim
    | coe s_2 =>
      have hs : s_1  <=  s_2 := WithBot.coe_le_coe.1 h
      cases s_1 with
      | inl x =>
        cases s_2 with
        | inl x' => exact (inlMap A B).toElement_mono (Sum.inl_le_inl_iff.1 hs)
        | inr y' => exact (Sum.not_inl_le_inr hs).elim
      | inr y =>
        cases s_2 with
        | inl x' => exact (Sum.not_inr_le_inl hs).elim
        | inr y' => exact (inrMap A B).toElement_mono (Sum.inr_le_inr_iff.1 hs)

/-- **Separated-sum domain iso (1982).** `|A + B| ?o WithBot (|A| ? |B|)`. Classical. -/
noncomputable def sumDomainIso :
    WithBot (A.Element ? B.Element) ?o (sumSystem A B).Element where
  toFun := assembleSum A B
  invFun := classifySum A B
  left_inv := classifySum_assembleSum A B
  right_inv := assembleSum_classifySum A B
  map_rel_iff' := by
    intro w_1 w_2
    constructor
    * intro h
      cases w_1 with
      | bot => exact bot_le
      | coe s_1 =>
        cases w_2 with
        | bot =>
          cases s_1 with
          | inl x =>
            exact False.elim (not_left_mem_sum_botElement A B
              (h (left_bot_mem_inlMap_toElement A B x)))
          | inr y =>
            exact False.elim (not_right_mem_sum_botElement A B
              (h (right_bot_mem_inrMap_toElement A B y)))
        | coe s_2 =>
          refine WithBot.coe_le_coe.2 ?_
          cases s_1 with
          | inl x =>
```

Lean 4 source (lines 401–469)

```
            cases s_2 with
            | inl x' => exact Sum.inl_le_inl_iff.2 ((inlMap_toElement_le_iff A B).1 h)
            | inr y' =>
              exact False.elim <|
                not_mem_right_of_mem_left A B ((inrMap A B).toElement y')
                  (h (left_bot_mem_inlMap_toElement A B x))
                  (right_bot_mem_inrMap_toElement A B y')
          | inr y =>
            cases s_2 with
            | inl x' =>
              exact False.elim <|
                not_mem_right_of_mem_left A B ((inlMap A B).toElement x')
                  (left_bot_mem_inlMap_toElement A B x')
                  (h (right_bot_mem_inrMap_toElement A B y))
            | inr y' => exact Sum.inr_le_inr_iff.2 ((inrMap_toElement_le_iff A B).1 h)
    * exact assembleSum_mono A B
```

```
/-! ## Function space `|A  ->  B| ?o ApproximableMap A B` -/

/-- Pointwise relation order on approximable maps (Prop 5.3 `Le`). -/
instance instPartialOrderApproximableMap : PartialOrder (ApproximableMap A B) where
  le := @Le _ _ _ _ A B
  le_refl _ _ _ h := h
  le_trans _ _ _ hfg hgh _ _ hf := hgh (hfg hf)
  le_antisymm _ _ hfg hgf := ApproximableMap.ext fun _ _ => <fun h => hfg h, fun h => hgf h>

theorem approxMap_toElement_le_iff {f g : ApproximableMap A B} :
    approxMap_toElement A B f  <=  approxMap_toElement A B g  <->  f  <=  g := by
  constructor
  * intro h u v hrel
    have hp : mkFunToken A B u v (f.rel_dom hrel) (f.rel_cod hrel)  in
        (approxMap_toElement A B f).carrier :=
      (mem_approxMap_toElement A B f).2 hrel
    exact (mem_approxMap_toElement A B g).1 (h hp)
  * intro hfg p hp
    exact (mem_approxMap_toElement A B g).2 (hfg ((mem_approxMap_toElement A B f).1 hp))

/-- **Function-space domain iso (1982, Thm 7.2).** `|A  ->  B| ?o ApproximableMap A B`. -/
noncomputable def functionSpaceDomainIso :
    ApproximableMap A B ?o (functionSystem A B).Element where
  toFun := approxMap_toElement A B
  invFun := element_toApproxMap A B
  left_inv := element_toApproxMap_approxMap_toElement A B
  right_inv := approxMap_toElement_element_toApproxMap A B
  map_rel_iff' := by
    intro f g
    exact approxMap_toElement_le_iff A B

end InfoSysConstructions

/-- Blueprint-facing name for the 1982 product domain isomorphism. -/
noncomputable abbrev infoSys_product_domain_equiv {a b : Type*} [DecidableEq a] [DecidableEq b]
    (A : InfoSys a) (B : InfoSys b) :
    A.Element  x  B.Element ?o (productSystem A B).Element :=
  InfoSysConstructions.productDomainIso A B

/-- Blueprint-facing name for the 1982 separated-sum domain isomorphism (classical). -/
noncomputable abbrev infoSys_sum_domain_equiv {a b : Type*} [DecidableEq a] [DecidableEq b]
    (A : InfoSys a) (B : InfoSys b) :
    WithBot (A.Element ? B.Element) ?o (sumSystem A B).Element :=
  InfoSysConstructions.sumDomainIso A B

/-- Blueprint-facing name for the 1982 function-space domain isomorphism. -/
noncomputable abbrev infoSys_function_space_domain_equiv {a b : Type*}
    [DecidableEq a] [DecidableEq b] (A : InfoSys a) (B : InfoSys b) :
    ApproximableMap A B ?o (functionSystem A B).Element :=
  InfoSysConstructions.functionSpaceDomainIso A B

end ScottModels
```

A.9 ScottModels/ScottMapBridge.lean

190 lines.

Lean 4 source

```
/-
Copyright (c) 2026  Lars Warren Ericson.  All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
```

```
Github:  https://github.com/catskillsresearch/scott_models
-/

import Mathlib.Order.Hom.Basic
import Scott1972.ContinuousLattice.FunctionSpaces
import Scott1982.Factoid46
import Scott1982.Factoid36
import Scott1982.Proposition53
import ScottModels.InfoSysConstructions
import ScottModels.PresentationDomains

/-!
# Construction cross-links -- ApproximableMap  <->  ScottContinuous  <->  ScottMap

* **Factoid 4.6:** `ApproximableMap A B ?o ScottContinuous A B` (1982).
* **1972 transport:** `ScottMap D D'` conjugates along `presentation_domains_equiv`
  (or the round-filter iso) to pointwise-ordered maps on the round presentation.
* Blueprint packaging: `infoSys_constructions_equiv` bundles the three 1982 domain
  isos with these cross-links.
-/

namespace ScottModels

open Scott1982
open Scott1982.Constructive
open Scott1982.InfoSys
open Scott1982.InfoSys.ApproximableMap
open Scott1972.ContinuousLattice
open ContinuousLatticeToNeighborhood

/-! ## Factoid 4.6 as an order isomorphism -/

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

theorem scottContinuous_ext {A : InfoSys a} {B : InfoSys b}
    {f g : ScottContinuous A B} (h : forall  x, f.toFun x = g.toFun x) : f = g := by
  obtain <tf, mf, df> := f
  obtain <tg, mg, dg> := g
  have : tf = tg := funext h
  subst this
  rfl

/-- Pointwise order on Scott-continuous maps of InfoSys domains. -/
instance instPartialOrderScottContinuous (A : InfoSys a) (B : InfoSys b) :
    PartialOrder (ScottContinuous A B) where
  le f g := forall  x, f.toFun x  <=  g.toFun x
  le_refl _ _ := le_refl _
  le_trans _ _ _ hfg hgh x := le_trans (hfg x) (hgh x)
  le_antisymm _ _ hfg hgf :=
    scottContinuous_ext fun x => le_antisymm (hfg x) (hgf x)

theorem ofScottContinuous_toScottContinuous {A : InfoSys a} {B : InfoSys b}
    (f : ApproximableMap A B) :
    ofScottContinuous (toScottContinuous f) = f := by
  refine ApproximableMap.ext fun u v => ?_
  constructor
  * intro <hu, hv, hsub>
    exact (f.rel_iff_closure_le hu hv).2
      (B.closure_le_element (f.toElement (A.closure u hu)) hv hsub)
  * intro hrel
    refine <f.rel_dom hrel, f.rel_cod hrel, ?_>
    intro y hy
    have hv : v  in  B.Con := f.rel_cod hrel
```

```
    have hu : u  in  A.Con := f.rel_dom hrel
    exact (f.rel_iff_closure_le hu hv).1 hrel (B.subset_closure hv (Finset.mem_coe.1 hy))

theorem toScottContinuous_ofScottContinuous {A : InfoSys a} {B : InfoSys b}
    (g : ScottContinuous A B) :
    toScottContinuous (ofScottContinuous g) = g :=
  scottContinuous_ext fun x => toElement_ofScottContinuous g x

/-- **Factoid 4.6.** Approximable maps  <->  Scott-continuous maps of domains. -/
noncomputable def approximableMap_scottContinuous_equiv (A : InfoSys a) (B : InfoSys b) :
    ApproximableMap A B ?o ScottContinuous A B where
  toFun := toScottContinuous
  invFun := ofScottContinuous
  left_inv := ofScottContinuous_toScottContinuous
  right_inv := toScottContinuous_ofScottContinuous
  map_rel_iff' := by
    intro f g
    exact (le_iff_toElement_le f g).symm

/-! ## ScottMap conjugation along the round presentation -/

variable {D E : Type*} [CompleteLattice D] [CompleteLattice E]

/-- Conjugate a Scott map along order isomorphisms of the underlying lattices. -/
noncomputable def conjScottMapFun {D' E' : Type*} [LE D'] [LE E']
    (?D : D ?o D') (?E : E ?o E') (f : ScottMap D E) : D'  ->  E' :=
  ??E  comp  (f : D  ->  E)  comp  ??D.symm

theorem conjScottMapFun_le_iff {D' E' : Type*} [PartialOrder D'] [PartialOrder E']
    (?D : D ?o D') (?E : E ?o E') {f g : ScottMap D E} :
    (forall  x : D', conjScottMapFun ?D ?E f x  <=  conjScottMapFun ?D ?E g x)  <->  f  <=  g
    := by
  constructor
  * intro h
    rw [ScottMap.le_def]
    intro x
    have hx : ?E (f x)  <=  ?E (g x) := by
      simpa [conjScottMapFun, OrderIso.symm_apply_apply] using h (?D x)
    exact ?E.map_rel_iff.mp hx
  * intro hfg x
    exact ?E.monotone (hfg (?D.symm x))

/-- Scott maps packaged as their conjugates along given presentation isos. -/
structure ConjScottMap {D' E' : Type*} [PartialOrder D'] [PartialOrder E']
    (?D : D ?o D') (?E : E ?o E') where
  scott : ScottMap D E

namespace ConjScottMap

variable {D' E' : Type*} [PartialOrder D'] [PartialOrder E']
variable (?D : D ?o D') (?E : E ?o E')

noncomputable instance : CoeFun (ConjScottMap ?D ?E) (fun _ => D'  ->  E') where
  coe g := conjScottMapFun ?D ?E g.scott

theorem ext {f g : ConjScottMap ?D ?E} (h : f.scott = g.scott) : f = g := by
  cases f; cases g; congr

noncomputable instance : PartialOrder (ConjScottMap ?D ?E) where
  le f g := forall  x, conjScottMapFun ?D ?E f.scott x  <=  conjScottMapFun ?D ?E g.scott x
  le_refl _ _ := le_refl _
  le_trans _ _ _ hfg hgh x := le_trans (hfg x) (hgh x)
  le_antisymm f g hfg hgf := by
```

```
    refine ext ?D ?E ?_
    exact le_antisymm
      ((conjScottMapFun_le_iff ?D ?E).1 hfg)
      ((conjScottMapFun_le_iff ?D ?E).1 hgf)

/-- **1972  <->  round presentation:** Scott maps ? conjugates along the given isos. -/
noncomputable def orderIso : ScottMap D E ?o ConjScottMap ?D ?E where
  toFun f := <f>
  invFun g := g.scott
  left_inv _ := rfl
  right_inv _ := rfl
  map_rel_iff' := by
    intro f g
    exact conjScottMapFun_le_iff ?D ?E

end ConjScottMap

section ContinuousRoundFilter

variable (hD : IsContinuousLattice D) (hE : IsContinuousLattice E)
include hD hE

/-- Conjugation along round ` Up `-filters (no `DecidableEq` needed). -/
noncomputable abbrev scottMap_roundFilter_iso :
    ScottMap D E ?o
      ConjScottMap (continuousLattice_roundFilter_iso hD)
        (continuousLattice_roundFilter_iso hE) :=
  ConjScottMap.orderIso (continuousLattice_roundFilter_iso hD)
    (continuousLattice_roundFilter_iso hE)

end ContinuousRoundFilter

section ContinuousRoundInfoSys

variable (hD : IsContinuousLattice D) (hE : IsContinuousLattice E)
variable [DecidableEq D] [DecidableEq E]
include hD hE

/-- Conjugation along round InfoSys elements of the ` Up `-basis. -/
noncomputable abbrev scottMap_roundInfoSys_iso :
    ScottMap D E ?o
      ConjScottMap (presentation_domains_equiv hD) (presentation_domains_equiv hE) :=
  ConjScottMap.orderIso (presentation_domains_equiv hD) (presentation_domains_equiv hE)

end ContinuousRoundInfoSys

/-! ## Blueprint packaging

Bundled 1982 construction domain isos + 1972/1982 function-space cross-links. -/
namespace infoSys_constructions_equiv

noncomputable abbrev product := @infoSys_product_domain_equiv
noncomputable abbrev sum := @infoSys_sum_domain_equiv
noncomputable abbrev functionSpace := @infoSys_function_space_domain_equiv
noncomputable abbrev approximable_scottContinuous := @approximableMap_scottContinuous_equiv
noncomputable abbrev scottMap_roundFilter := @scottMap_roundFilter_iso
noncomputable abbrev scottMap_roundInfoSys := @scottMap_roundInfoSys_iso

end infoSys_constructions_equiv

end ScottModels
```

A.10 ScottModels/WorkedExampleSExpr.lean

139 lines.

Lean 4 source

```
/-
Copyright (c) 2026  Lars Warren Ericson.  All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github:  https://github.com/catskillsresearch/scott_models
-/

import Mathlib.Order.Hom.Basic
import Mathlib.Order.Hom.WithTopBot
import Mathlib.Data.Sum.Order
import Mathlib.Order.Ideal
import Scott1982.Factoid24
import Scott1982.Factoid81
import Scott1982.Proposition54
import ScottModels.InfoSysToNeighborhood
import ScottModels.InfoSysToIdealCompletion
import ScottModels.PresentationDomains
import ScottModels.InfoSysConstructions
import ScottModels.ScottMapBridge

/-!
# Worked example -- S-expression / tree domain `T  ~=  A + (T  x  T)`

Scott 1982 Factoid 8.1 (`treeSystem`) over the Nat lower-bound atom system
(Factoid 2.4), walked through this package?s bridges: information system  ->
neighbourhood filters  ->  ideal completion, plus identity approximable maps as
Scott-continuous maps (Factoid 4.6).
-/

namespace ScottModels

open Scott1982
open Scott1982.Constructive
open Scott1982.InfoSys
open Scott1982.InfoSys.ApproximableMap
open Order

/-! ## Atom system (Factoid 2.4, named) -/

/-- Scott?s Nat lower-bound information system (Factoid 2.4), as a named definition. -/
def lowerBoundSystem : InfoSys Nat where
  bot := 0
  Con := Set.univ
  Ent := Factoid24.lowerBoundEnt
  con_subset := by
    intro u v _ _
    exact Set.mem_univ v
  con_sing := by
    intro _
    exact Set.mem_univ _
  ent_con := by
    intro u a _
    exact Set.mem_univ _
  ent_bot := by
    intro u _
    exact Or.inl rfl
  ent_refl := by
    intro u a _ ha
```

```
    exact Or.inr <a, ha, le_rfl>
  ent_trans := by
    intro u v c _ _ hvEnt huEnt
    rcases huEnt with rfl | <n, hn, hcn>
    * exact Or.inl rfl
    * rcases hvEnt n hn with rfl | <k, hk, hnk>
      * exact Or.inl (Nat.le_zero.mp hcn)
      * exact Or.inr <k, hk, le_trans hcn hnk>

/-! ## S-expression system -/

/-- Tree / S-expression information system over lower-bound atoms. -/
abbrev SexSys : InfoSys (TreeToken Nat) :=
  treeSystem lowerBoundSystem

/-- Official right-hand side `A + (T  x  T)`. -/
abbrev SexRhs : InfoSys (SumToken Nat (ProdToken SexSys SexSys)) :=
  treeRhs lowerBoundSystem

theorem sexRhs_eq_sum_product :
    SexRhs = sumSystem lowerBoundSystem (productSystem SexSys SexSys) :=
  treeRhs_eq_sum_product lowerBoundSystem

/-- Unfolding tokens into the sum-of-product carrier (Factoid 8.1). -/
theorem sexUnfold_atom (n : Nat) :
    treeUnfold lowerBoundSystem (.atom n) = SumToken.left n :=
  treeUnfold_atom lowerBoundSystem n

/-! ## Concrete finite elements -/

/-- Closure of a singleton atom token `{atom n}`. -/
noncomputable def sexAtom (n : Nat) : SexSys.Element :=
  SexSys.closure {TreeToken.atom n} (SexSys.con_sing _)

theorem sexAtom_mem_self (n : Nat) : TreeToken.atom n  in  (sexAtom n).carrier :=
  SexSys.subset_closure (SexSys.con_sing _) (Finset.mem_singleton_self _)

/-! ## 1982  ->  1980: basic-open neighbourhood filters -/

/-- `|T| ?o` filters of basic opens `[u]`. -/
noncomputable abbrev sexNeighborhoodIso :
    SexSys.Element ?o (InfoSysToNeighborhood.toNeighborhoodSystem SexSys).Element :=
  InfoSysToNeighborhood.domainOrderIso SexSys

/-! ## 1982  ->  ideal completion -/

/-- `|T| ?o Ideal (FiniteElement T)`. -/
noncomputable abbrev sexIdealIso :
    SexSys.Element ?o Ideal (InfoSysToIdealCompletion.FiniteElement SexSys) :=
  InfoSysToIdealCompletion.domainOrderIso SexSys

/-- Neighbourhood filters ? ideals of finite elements (constructive triangle). -/
noncomputable abbrev sexNeighborhoodIdealIso :=
  neighborhood_ideal_iso SexSys

/-! ## Domain equation at the level of domains (1982 constructions) -/

/--
`WithBot (|A| ? (|T|  x  |T|)) ?o |A + (T  x  T)|`, composing this package?s product and
separated-sum isos with Factoid 8.1?s RHS.
-/
noncomputable def sexDomainEquationIso :
    WithBot (lowerBoundSystem.Element ? (SexSys.Element  x  SexSys.Element)) ?o
```

```
      SexRhs.Element :=
  let ?Prod := InfoSysConstructions.productDomainIso SexSys SexSys
  let ?Sum := InfoSysConstructions.sumDomainIso lowerBoundSystem (productSystem SexSys SexSys)
  let mid :=
    (OrderIso.refl lowerBoundSystem.Element).sumCongr ?Prod |>.withBotCongr
  mid.trans ?Sum

/-! ## Morphisms: identity is Scott-continuous (Factoid 4.6) -/

/-- Identity approximable map on `T`, as a Scott-continuous endomap. -/
noncomputable abbrev sexIdScottContinuous : ScottContinuous SexSys SexSys :=
  (approximableMap_scottContinuous_equiv SexSys SexSys) (idMap SexSys)

theorem sexId_toElement (x : SexSys.Element) :
    sexIdScottContinuous.toFun x = x :=
  idMap_toElement SexSys x

end ScottModels
```

A.11 ScottModels/Equivalence.lean

41 lines.

Lean 4 source

```
/-
Copyright (c) 2026  Lars Warren Ericson.  All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github:  https://github.com/catskillsresearch/scott_models
-/

import ScottModels.NeighborhoodToInfoSys
import ScottModels.InfoSysToNeighborhood
import ScottModels.ContinuousLatticeToNeighborhood
import ScottModels.InfoSysToIdealCompletion
import ScottModels.IdealCompletionToContinuousLattice
import ScottModels.PresentationDomains
import ScottModels.InfoSysConstructions
import ScottModels.ScottMapBridge

/-!
# Equivalence theorems

Bridges between Scott's 1972 continuous-lattice presentation, 1980 neighbourhood systems
(PRG-19), and 1982 information systems. Sibling packages are finished; this module holds
the cross-presentation maps. See `arxiv.md` / `HANDOFF.md`.

## Status

* `neighborhoodSystem_to_infoSys` -- **done** (`NeighborhoodToInfoSys.lean`)
* `infoSys_to_neighborhoodSystem` -- **done** (`InfoSysToNeighborhood.lean`)
* `continuousLattice_to_neighborhoodSystem` -- **done** (`ContinuousLatticeToNeighborhood.lean
    `;
  `D ?o RoundFilter`, not raw `|?|`)
* `infoSys_to_idealCompletion` -- **done** (`InfoSysToIdealCompletion.lean`;
  `|A| ?o Ideal (FiniteElement A)`)
* `idealCompletion_to_continuousLattice` -- **done** (`IdealCompletionToContinuousLattice.lean
    `;
  algebraic complete lattice  =>  `IsContinuousLattice`; classical ` << `)
* `presentation_domains_equiv` -- **done** (`PresentationDomains.lean`;
  `D ?o RoundFilter ?o RoundInfoSysElement` via `wayBelowNbhdBasis`)
```

```
* `infoSys_constructions_equiv` -- **done** (`InfoSysConstructions.lean` +
  `ScottMapBridge.lean`; 1982 product/sum/function-space domain isos;
  `ApproximableMap ?o ScottContinuous`; `ScottMap` conjugates along round presentation)
* worked example -- **done** (`WorkedExampleSExpr.lean`; Factoid 8.1 trees through
  neighbourhood / ideal bridges + domain-equation iso)
-/
```